

PerformanceReviewsAnalyser_31

December 31, 2023

```
[ ]: from google.colab import drive
drive.mount('/content/drive')
```

Mounted at /content/drive

1 Importing necessary libraries

```
[ ]: # Install vaderSentiment
!pip install vaderSentiment
!pip install tqdm
import random
import pandas as pd

# Import SentimentIntensityAnalyzer
from vaderSentiment.vaderSentiment import SentimentIntensityAnalyzer
from tqdm.notebook import tqdm
```

Requirement already satisfied: vaderSentiment in /usr/local/lib/python3.10/dist-packages (3.3.2)

Requirement already satisfied: requests in /usr/local/lib/python3.10/dist-packages (from vaderSentiment) (2.31.0)

Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.10/dist-packages (from requests->vaderSentiment) (3.3.2)

Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.10/dist-packages (from requests->vaderSentiment) (3.6)

Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.10/dist-packages (from requests->vaderSentiment) (2.0.7)

Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.10/dist-packages (from requests->vaderSentiment) (2023.11.17)

Requirement already satisfied: tqdm in /usr/local/lib/python3.10/dist-packages (4.66.1)

```
[ ]: # @title Default title text
```

```
# Names from England, Greece, India, and Israel
```

```

english_first_names = ["Oliver", "Harry", "Amelia", "Olivia", "George",
    ↳ "Isabella", "Charlie", "Sophia", "Jack", "Emily", "Henry", "Lily", "Leo",
    ↳ "Grace"]
greek_first_names = ["Giannis", "Nikos", "Eleni", "Kostas", "Dimitris",
    ↳ "Maria", "Sofia", "Vasilis", "Alexandra", "Andreas", "Yiorgos", "Anastasia",
    ↳ "Panos", "Irene"]
indian_first_names = ["Aarav", "Vivaan", "Aditi", "Aanya", "Saanvi", "Advait",
    ↳ "Ishaan", "Ananya", "Kabir", "Diya", "Arjun", "Siya", "Rohan", "Myra"]
israeli_first_names = ["Noah", "Liam", "Abigail", "Emma", "Ethan", "Sofia",
    ↳ "Daniel", "Mia", "David", "Ava", "Moshe", "Hannah", "Ariel", "Sarah"]

english_last_names = ["Smith", "Jones", "Taylor", "Brown", "Johnson",
    ↳ "Williams", "Miller", "Wilson", "Moore", "Anderson", "Thomas", "Jackson",
    ↳ "White", "Harris"]
greek_last_names = ["Papadopoulos", "Nikolaidis", "Georgiou", "Dimitriou",
    ↳ "Kostas", "Papadakis", "Vasiliou", "Christou", "Angelopoulos",
    ↳ "Michalopoulos", "Papathanasiou", "Karakostas", "Economou", "Stavropoulos"]
indian_last_names = ["Kumar", "Sharma", "Singh", "Gupta", "Patel", "Mehta",
    ↳ "Jain", "Reddy", "Shah", "Khan", "Ali", "Desai", "Chatterjee", "Bose"]
israeli_last_names = ["Cohen", "Levi", "Mizrahi", "Peretz", "Friedman", "Katz",
    ↳ "Goldberg", "Levin", "Gross", "Weiss", "Halevi", "Meir", "Shapira",
    ↳ "Ben-David"]

# Define names by ethnicity
english_names = [(first, last) for first in english_first_names for last in
    ↳ english_last_names]
greek_names = [(first, last) for first in greek_first_names for last in
    ↳ greek_last_names]
indian_names = [(first, last) for first in indian_first_names for last in
    ↳ indian_last_names]
israeli_names = [(first, last) for first in israeli_first_names for last in
    ↳ israeli_last_names]

# Combine all names while maintaining ethnicity
all_names = english_names + greek_names + indian_names + israeli_names

# Positions typical in companies like Google
positions = [
    "Software Engineer", "Senior Software Engineer",
    "Product Manager", "Associate Product Manager",
    "Data Scientist", "Data Analyst",
    "User Experience Designer", "Graphic Designer",
    "Sales Engineer", "Sales Representative",
    "Technical Program Manager", "Project Manager",
    "Research Scientist", "Research Assistant",
    "Business Analyst", "Business Consultant",
    "Marketing Specialist", "Digital Marketing Manager",
    "HR Manager", "Recruiter"
]

```

```

]

# Expanded positions and their corresponding departments
positions_to_department = {
    "Software Engineer": "Engineering",
    "Senior Software Engineer": "Engineering",
    "Product Manager": "Product Management",
    "Associate Product Manager": "Product Management",
    "Data Scientist": "Data Science",
    "Data Analyst": "Data Science",
    "User Experience Designer": "Design",
    "Graphic Designer": "Design",
    "Sales Engineer": "Sales",
    "Sales Representative": "Sales",
    "Technical Program Manager": "Technical Program Management",
    "Project Manager": "Technical Program Management",
    "Research Scientist": "Research",
    "Research Assistant": "Research",
    "Business Analyst": "Business Analysis",
    "Business Consultant": "Business Analysis",
    "Marketing Specialist": "Marketing",
    "Digital Marketing Manager": "Marketing",
    "HR Manager": "Human Resources",
    "Recruiter": "Human Resources"
}

# Sample manager comments
manager_comments = [
    "{} shows outstanding problem-solving skills ", "{} needs to work on their_
↪communication ",
    "{} is an exceptional team player ", "{} could improve their technical_
↪expertise",
    "{} demonstrates excellent leadership ", "{} lacks initiative in team_
↪projects",
    "{} is highly creative and innovative in tasks ", "{} struggles with time_
↪management ",
    "{} consistently exceeds performance targets", "{} needs to enhance their_
↪client management skills",
    "{} displays a strong work ethic and professionalism", "{} has difficulties_
↪adapting to new technologies",
    "{} is a true asset in collaborative projects", "{} demonstrates impressive_
↪learning and growth potential",
    "{} is always ready to take on new challenges", "{} shows resilience under_
↪pressure",
    "{} excels in delivering detailed and accurate work", "{} could benefit_
↪from more teamwork opportunities",

```

```

    "{} has a keen eye for innovation and improvement", "{} is exceptional in
↳mentoring and coaching others"
]
comment_to_theme = {
    "problem-solving": ["technical_skills", "innovation"],
    "communication": ["communication"],
    "team player": ["teamwork", "leadership"],
    "technical expertise": ["technical_skills"],
    "leadership": ["leadership", "work_ethic"],
    "initiative": ["innovation", "work_ethic"],
    "creative": ["innovation"],
    "time management": ["time_management"],
    "performance targets": ["work_ethic", "technical_skills"],
    "client management": ["client_management"],
    "work ethic": ["work_ethic"],
    "new technologies": ["adaptability", "technical_skills"],
    "collaborative projects": ["teamwork", "communication"],
    "learning and growth": ["learning_growth"],
    "new challenges": ["adaptability", "innovation"],
    "under pressure": ["resilience", "work_ethic"],
    "detailed and accurate work": ["technical_skills"],
    "teamwork opportunities": ["teamwork"],
    "innovation and improvement": ["innovation", "adaptability"],
    "mentoring and coaching": ["leadership", "learning_growth"]
}

def generate_years_of_experience(age):
    if age < 25:
        return random.randint(0, age - 18) # For ages 21-24, experience ranges
↳from 0 to age-18
    elif 25 <= age < 30:
        return random.randint(2, 7) # 2-7 years for ages 25-29
    elif 30 <= age < 40:
        return random.randint(5, 15) # 5-15 years for ages 30-39
    elif 40 <= age < 50:
        return random.randint(10, 25) # 10-25 years for ages 40-49
    else:
        return random.randint(15, 35) # 15-35 years for ages 50-60

# Function to randomly select a name and determine its ethnicity
def select_name_and_determine_ethnicity(prefer_english=False,
↳prefer_greek=True, prefer_indian=False, prefer_israeli=False):
    ethnicity_weights = {
        'English': 3 if prefer_english else 1,
        'Greek': 3 if prefer_greek else 1,
        'Indian': 3 if prefer_indian else 1,
        'Israeli': 3 if prefer_israeli else 1
    }

```

```

}

# Randomly choose an ethnicity based on the specified probabilities
ethnicity = random.choices(
    population=list(ethnicity_weights.keys()),
    weights=list(ethnicity_weights.values()),
    k=1
)[0]

first_names, last_names = {
    'English': (english_first_names, english_last_names),
    'Greek': (greek_first_names, greek_last_names),
    'Indian': (indian_first_names, indian_last_names),
    'Israeli': (israeli_first_names, israeli_last_names)
}[ethnicity]

first_name = random.choice(first_names)
last_name = random.choice(last_names)
full_name = f"{first_name} {last_name}"
return full_name, ethnicity

# Categorize each manager comment
def categorize_comment(comment):
    analyzer = SentimentIntensityAnalyzer()
    sentiment_score = analyzer.polarity_scores(comment)
    return 'positive' if sentiment_score['compound'] >= 0.05 else 'negative'

positive_comments = [comment for comment in manager_comments if
    ↪categorize_comment(comment) == 'positive']
negative_comments = [comment for comment in manager_comments if
    ↪categorize_comment(comment) == 'negative']

# Ensuring unique combinations of name, surname, and manager comment
unique_combinations = set()
while len(unique_combinations) < 3000:
    name, ethnicity = select_name_and_determine_ethnicity(prefer_greek=True)
    comment = random.choice(manager_comments).format(name)
    unique_combinations.add((name, ethnicity, comment))

# Expanded themes for comprehensive employee reviews
expanded_themes = {
    # Positive themes
    "positive_teamwork": [
        "{} consistently contributes positively to team projects",
        "{} excels in collaboration and team coordination",
        "{} is an inspiring team leader who motivates others",
    ]
}

```

```

    "{} fosters a positive team environment",
    "{} is adept at conflict resolution within the team",
    "{} often goes above and beyond for team success",
    "{} shows exceptional empathy and understanding towards team members",
    "{} consistently brings out the best in their teammates",
    "{} is a natural at building team spirit",
    "{} actively contributes to team brainstorming sessions",
    "{} excels in creating a collaborative team culture",
    "{} is a key player in team initiatives",
    "{} leads by example in team settings",
    "{} has a talent for uniting team members towards a common goal",
    "{} is highly respected and valued by their team"
],

# Negative themes
"negative_teamwork": [
    "{} needs to improve in teamwork and collaboration",
    "{} struggles in team settings and often works in isolation",
    "{} has difficulty accepting feedback from team members",
    "{} tends to dominate team discussions rather than listening",
    "{} has been uncooperative in recent team projects",
    "{} shows reluctance in sharing tasks with other team members",
    "{} often misses team meetings or arrives unprepared",
    "{} struggles with delegating tasks, affecting team efficiency",
    "{} has been resistant to team changes and new ideas",
    "{} needs to work on building trust within the team",
    "{} has a tendency to criticize rather than support team members",
    "{} needs to develop better conflict management skills in the team",
    "{} has shown a lack of commitment to team goals",
    "{} often underestimates the value of teamwork",
    "{} has struggled to maintain positive relationships within the team"
],

# Neutral themes
"neutral_teamwork": [
    "{} is generally a good team player and participates actively",
    "{} is consistent in their teamwork but has room for improvement",
    "{} attends team meetings and contributes as needed",
    "{} is cooperative in team projects but tends to follow rather than_
↳lead",
    "{} maintains a neutral stance in team discussions",
    "{} completes assigned team tasks on time",
    "{} is a reliable member of the team, though not a standout",
    "{} supports team decisions but rarely initiates action",
    "{} is agreeable in team settings but lacks proactive engagement",
    "{} performs adequately in team environments",
    "{} contributes to team projects but lacks enthusiasm",

```

```

        "{} is consistent in team participation but not a key influencer",
        "{} tends to stay in the background during team activities",
        "{} is a steady team member but lacks dynamism",
        "{} meets basic expectations in teamwork but lacks initiative"
    ],
    "positive_leadership": [
        "{} inspires their team with a clear vision",
        "{} effectively delegates tasks and empowers team members",
        "{} consistently recognizes and appreciates team members'
↳contributions",
        "{} leads with integrity and earns respect from the team",
        "{} effectively mentors and develops emerging leaders",
        "{} maintains high morale and team spirit even under pressure",
        "{} navigates team challenges with excellent problem-solving skills",
        "{} fosters an inclusive and diverse team environment",
        "{} consistently leads by example and sets high standards",
        "{} communicates a clear and compelling direction for the team",
        "{} builds strong relationships with team members based on trust",
        "{} drives the team towards exceptional performance",
        "{} creates opportunities for team members to grow and succeed",
        "{} listens actively and encourages open communication",
        "{} is adaptable and responsive to team feedback"
    ],

    # Negative leadership
    "negative_leadership": [
        "{} often micromanages, hindering team autonomy",
        "{} struggles to effectively communicate with the team",
        "{} rarely acknowledges or rewards team members' efforts",
        "{} displays favoritism, affecting team morale",
        "{} is often indecisive, causing delays in team progress",
        "{} avoids addressing conflicts, leading to unresolved issues",
        "{} lacks transparency, creating uncertainty in the team",
        "{} struggles with providing constructive feedback",
        "{} has difficulty inspiring or motivating the team",
        "{} is resistant to new ideas or approaches suggested by the team",
        "{} often overlooks the need for team development",
        "{} has a rigid leadership style that stifles creativity",
        "{} struggles to build trust within the team",
        "{} rarely engages in active listening, missing valuable input",
        "{} has a top-down approach that limits team collaboration"
    ],

    # Neutral leadership
    "neutral_leadership": [
        "{} maintains a consistent leadership style",
        "{} meets basic expectations as a team leader",

```

```

    "{} communicates team objectives but lacks inspirational qualities",
    "{} is a functional leader but lacks charisma",
    "{} maintains order in the team without being overly innovative",
    "{} has a conventional approach to leadership",
    "{} focuses on task completion without much focus on team building",
    "{} is seen as a manager rather than a leader by the team",
    "{} applies standard leadership techniques effectively",
    "{} ensures team compliance with policies and procedures",
    "{} is more focused on processes than people in the team",
    "{} tends to stick to traditional leadership methods",
    "{} is reliable but not particularly dynamic as a leader",
    "{} follows a set routine in managing the team",
    "{} focuses on maintaining the status quo in team management"
],

# Positive Technical Skills
"positive_technical_skills": [
    "{} demonstrates exceptional proficiency in their technical skills",
    "{} consistently applies technical knowledge effectively in projects",
    "{} shows innovative problem-solving using technical skills",
    "{} excels in adapting technical skills to new technologies",
    "{} has a proven track record of technical excellence in their field",
    "{} consistently delivers high-quality technical solutions"
],

# Negative Technical Skills
"negative_technical_skills": [
    "{} needs to enhance their technical skills in specific areas",
    "{} struggles to apply technical knowledge in practical scenarios",
    "{} often requires assistance with technical tasks",
    "{} has difficulty keeping up with current technical trends",
    "{} shows limited understanding of advanced technical concepts",
    "{} tends to avoid technically challenging tasks"
],

# Neutral Technical Skills
"neutral_technical_skills": [
    "{} has a basic understanding of technical skills",
    "{} applies technical skills as needed but lacks depth",
    "{} is competent in technical tasks but lacks innovation",
    "{} demonstrates average technical competence",
    "{} has the potential to improve technical skills with guidance",
    "{} performs routine technical tasks effectively"
],

# Positive Communication
"positive_communication": [

```



```

    "{} excels in clear and effective communication",
    "{} is highly skilled in both written and verbal communication",
    "{} effectively tailors communication to different audiences",
    "{} demonstrates excellent persuasive communication skills",
    "{} is an articulate speaker and thoughtful listener",
    "{} excels in communicating complex ideas simply and effectively"
],

# Negative Communication
"negative_communication": [
    "{} needs to improve clarity and conciseness in communication",
    "{} struggles with effective verbal communication",
    "{} often misinterprets written instructions",
    "{} tends to communicate in a vague or ambiguous manner",
    "{} shows difficulty in expressing ideas clearly to others",
    "{} has a tendency to avoid direct communication"
],

# Neutral Communication
"neutral_communication": [
    "{} communicates adequately but lacks engagement",
    "{} is sometimes unclear in conveying complex ideas",
    "{} meets basic communication standards",
    "{} tends to be passive in communication situations",
    "{} is clear in communication but lacks persuasion skills",
    "{} communicates effectively but lacks emotional depth"
],

# Positive Innovation
"positive_innovation": [
    "{} consistently introduces creative solutions",
    "{} is a driving force behind innovative ideas",
    "{} excels in thinking outside the box",
    "{} regularly challenges conventional thinking with novel ideas",
    "{} is known for pioneering innovative approaches",
    "{} effectively integrates innovation with practical application"
],

# Negative Innovation
"negative_innovation": [
    "{} rarely contributes new ideas",
    "{} is hesitant to embrace new methodologies",
    "{} lacks initiative in innovation",
    "{} shows resistance to exploring unorthodox solutions",
    "{} tends to dismiss innovative ideas too quickly",
    "{} prefers conventional methods over innovative approaches"
],

```

```

# Neutral Innovation
"neutral_innovation": [
    "{} occasionally suggests new ideas",
    "{} adapts to innovative changes but does not lead them",
    "{} shows average interest in innovative approaches",
    "{} is open to innovation but lacks active involvement",
    "{} acknowledges innovative ideas but rarely implements them",
    "{} is cautious but fair in evaluating new ideas"
],

# Positive Adaptability
"positive_adaptability": [
    "{} adapts quickly to new situations and challenges",
    "{} shows great flexibility in changing environments",
    "{} is adept at handling unexpected changes",
    "{} thrives under dynamic and evolving conditions",
    "{} seamlessly adjusts strategies to meet new demands",
    "{} is skilled at turning challenges into opportunities"
],

# Negative Adaptability
"negative_adaptability": [
    "{} struggles with adapting to new processes",
    "{} is resistant to change and new ideas",
    "{} finds it challenging to adjust to different situations",
    "{} often becomes overwhelmed by changes in routine",
    "{} shows inflexibility when faced with new methods",
    "{} tends to be slow in adapting to organizational changes"
],

# Neutral Adaptability
"neutral_adaptability": [
    "{} adjusts to new situations as required",
    "{} is moderately flexible in new environments",
    "{} shows some resistance but generally adapts well",
    "{} can adapt but often needs time and encouragement",
    "{} is open to change but lacks enthusiasm for it",
    "{} manages to keep up with changes but doesn't lead the way"
],

# Positive Work Ethic
"positive_work_ethic": [
    "{} demonstrates a strong commitment to their work",
    "{} consistently shows dedication and diligence",
    "{} has an exemplary work ethic",
    "{} is known for going the extra mile in their tasks",

```

```

    "{} maintains a high level of professionalism at all times",
    "{} sets a strong example of productivity and commitment"
],

# Negative Work Ethic
"negative_work_ethic": [
    "{} often lacks motivation and drive in their work",
    "{} needs to improve their dedication and focus",
    "{} shows inconsistency in their work ethic",
    "{} tends to procrastinate and miss deadlines",
    "{} lacks the initiative to take on new challenges",
    "{} shows minimal effort in tasks outside their comfort zone"
],

# Neutral Work Ethic
"neutral_work_ethic": [
    "{} generally maintains a good work ethic",
    "{} is usually reliable but has moments of inconsistency",
    "{} works well but lacks extra dedication",
    "{} completes tasks but rarely goes beyond basic requirements",
    "{} is steady and dependable but not exceptionally proactive",
    "{} consistently meets expectations but seldom exceeds them"
],

# Positive Client Management
"positive_client_management": [
    "{} excels in building strong relationships with clients",
    "{} demonstrates exceptional client management skills",
    "{} consistently receives positive feedback from clients",
    "{} skillfully navigates client needs and expectations",
    "{} is adept at resolving client issues effectively",
    "{} builds lasting rapport with clients"
],

# Negative Client Management
"negative_client_management": [
    "{} needs to improve in managing client relationships",
    "{} struggles with effective communication with clients",
    "{} has received negative feedback from clients",
    "{} often overlooks client concerns and feedback",
    "{} has difficulty in maintaining consistent client interactions",
    "{} shows a lack of understanding of client perspectives"
],

# Neutral Client Management
"neutral_client_management": [
    "{} manages client relationships adequately",

```

```

    "{} maintains basic client satisfaction",
    "{} is competent in client management but lacks exceptional skills",
    "{} meets client requirements but doesn't exceed expectations",
    "{} maintains professional relations but lacks client engagement",
    "{} is consistent but lacks the personal touch in client interactions"
],

# Positive Time Management
"positive_time_management": [
    "{} excels in managing their time efficiently",
    "{} consistently meets deadlines and manages tasks effectively",
    "{} has exceptional organizational skills",
    "{} is adept at prioritizing tasks and managing workload",
    "{} demonstrates excellent planning and scheduling abilities",
    "{} consistently uses time productively to achieve goals"
],

# Negative Time Management
"negative_time_management": [
    "{} often struggles with meeting deadlines",
    "{} needs to improve in organizing and prioritizing tasks",
    "{} has difficulty managing time effectively",
    "{} frequently underestimates the time required for tasks",
    "{} is often late in delivering assignments",
    "{} has a tendency to get sidetracked by minor details"
],

# Neutral Time Management
"neutral_time_management": [
    "{} generally meets deadlines but can improve time management",
    "{} is capable of managing tasks but lacks efficiency",
    "{} is average in time management skills",
    "{} manages to complete tasks on time but often at the last minute",
    "{} is organized but can struggle with unexpected changes",
    "{} meets most deadlines but could utilize time more effectively"
],

# Positive Learning And Growth
"positive_learning_growth": [
    "{} is highly committed to personal and professional growth",
    "{} actively seeks out learning opportunities",
    "{} demonstrates rapid learning and skill development",
    "{} eagerly embraces new learning challenges",
    "{} consistently applies new knowledge to improve performance",
    "{} shows a strong drive for continuous self-improvement"
],

```

```

# Negative Learning And Growth
"negative_learning_growth": [
    "{} shows little interest in learning and self-improvement",
    "{} is slow to acquire new skills and knowledge",
    "{} needs to show more initiative in personal growth",
    "{} is reluctant to step out of their comfort zone to learn",
    "{} lacks curiosity in exploring new ideas and skills",
    "{} is indifferent to opportunities for professional development"
],

# Neutral Learning And Growth
"neutral_learning_growth": [
    "{} engages in learning but lacks enthusiasm",
    "{} is open to learning new things but is not proactive",
    "{} shows some interest in professional development",
    "{} learns new skills as needed but without much passion",
    "{} participates in training but doesn't seek additional learning",
    "{} shows a basic willingness to learn but lacks ambition for growth"
]
}

# Function to determine the sentiment of a comment
def determine_sentiment(comment):
    analyzer = SentimentIntensityAnalyzer()
    sentiment_score = analyzer.polarity_scores(comment)
    if sentiment_score['compound'] >= 0.05:
        return 'positive'
    elif sentiment_score['compound'] <= -0.05:
        return 'negative'
    else:
        return 'neutral'

# Function to generate performance review based on manager comment
def generate_linked_review(manager_comment):
    sentiment = determine_sentiment(manager_comment)
    relevant_themes = []

    # Select themes based on sentiment
    if sentiment == 'positive':
        theme_key = 'positive_{}'
    elif sentiment == 'negative':
        theme_key = 'negative_{}'
    else: # neutral
        theme_key = 'neutral_{}'

```

```

for key, themes in comment_to_theme.items():
    if key in manager_comment:
        for theme in themes:
            relevant_theme_key = theme_key.format(theme)
            if relevant_theme_key in expanded_themes:
                # Select two comments from the relevant theme
                relevant_comments = random.
↳sample(expanded_themes[relevant_theme_key], 2)
                relevant_themes.extend(relevant_comments)

if not relevant_themes: # If no specific themes are found, return None
    return None

detailed_comment = ". ".join(relevant_themes) + "."
return detailed_comment

# Function to generate a comprehensive performance review
def generate_comprehensive_review(employee_name):
    selected_comments = [random.choice(expanded_themes[theme]) for theme in_
↳expanded_themes]
    detailed_comment = ". ".join([comment.format(employee_name) for comment in_
↳selected_comments]) + "."
    return detailed_comment

# Flag to control biased or unbiased data generation
use_bias = True # Set to True for biased dataset, False for unbiased

# Generating data for 3000 employees
num_records = 3000
data = []
for _ in tqdm(range(num_records), desc="Generating Data"):
    name, ethnicity = select_name_and_determine_ethnicity()
    age = random.randint(21, 60) # Random age between 21 and 60

    if use_bias:
        # Biased comment selection
        if (ethnicity == "Greek" and 21 <= age <= 35) or \
            (ethnicity == "English" and 21 <= age <= 35) or \
            (ethnicity == "Indian" and 21 <= age <= 35) or \
            (ethnicity == "Israeli" and 21 <= age <= 35):
            comment = random.choice(positive_comments).format(name)
        else:
            comment = random.choice(negative_comments).format(name)
    else:
        # Unbiased comment selection

```

```

        comment = random.choice(manager_comments).format(name)

    id = random.randint(111111, 999999)
    employee_name_with_id = f"{name} ({id})"
    position = random.choice(list(positions_to_department.keys()))
    department = positions_to_department[position]
    years_of_experience = generate_years_of_experience(age)

    # Generate a linked review based on the manager's comment
    linked_review = generate_linked_review(comment)
    if linked_review:
        linked_review_formatted = ". ".join([comment.format(name) for comment_
↪in linked_review.split(". ")]) + "."
    else:
        linked_review_formatted = ""

    # Append additional comments from a comprehensive review
    #comprehensive_review = generate_comprehensive_review(name)
    review = linked_review_formatted
    #comprehensive_review = generate_comprehensive_review(name)
    #review = linked_review_formatted + " " + comprehensive_review if
↪linked_review else comprehensive_review

    data.append({
        "Employee Name": f"{name} ({id})",
        "Ethnicity": ethnicity,
        "Age": age,
        "Years of Experience": years_of_experience,
        "Manager Comment": comment,
        "Employee Performance Review": linked_review_formatted,
        "Position": position,
        "Department": department
    })

# Create DataFrame
df = pd.DataFrame(data)

# Save to Excel
local_path = 'DataPreprocessing.xlsx'
df.to_excel(local_path, index=False)
!mv DataPreprocessing.xlsx /content/drive/MyDrive/MasterNeapolis/

```

Generating Data: 0%| | 0/3000 [00:00<?, ?it/s]

Dataset Created and saved to One Drive

2 Importing more Libraries to Create the Dashboard

```
[ ]: # Check if the necessary import are already imported
try:
    import pandas as pd
except ImportError:

    import pandas as pd

try:
    from nltk.sentiment import SentimentIntensityAnalyzer
except ImportError:

    from nltk.sentiment import SentimentIntensityAnalyzer

try:
    import ipywidgets as widgets
except ImportError:

    import ipywidgets as widgets

try:
    from collections import Counter
except ImportError:

    import ipywidgets as widgets

try:
    from IPython.display import display, clear_output, HTML
except ImportError:

    from IPython.display import display, clear_output, HTML

try:
    from google.colab import files
except ImportError:

    from google.colab import files

try:
    import io
except ImportError:
```



```

import io

try:
    import re
except ImportError:

    import re

try:
    import numpy as np
except ImportError:

    import re

try:
    import matplotlib.pyplot as plt
except ImportError:

    !pip install matplotlib
    import matplotlib.pyplot as plt

try:
    from vaderSentiment.vaderSentiment import SentimentIntensityAnalyzer
except ImportError:
    !pip install vaderSentiment
    from vaderSentiment.vaderSentiment import SentimentIntensityAnalyzer

```

Collecting vaderSentiment

Downloading vaderSentiment-3.3.2-py2.py3-none-any.whl (125 kB)

126.0/126.0

kB 1.7 MB/s eta 0:00:00

Requirement already satisfied: requests in /usr/local/lib/python3.10/dist-packages (from vaderSentiment) (2.31.0)

Requirement already satisfied: charset-normalizer<4,>=2 in

/usr/local/lib/python3.10/dist-packages (from requests->vaderSentiment) (3.3.2)

Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.10/dist-packages (from requests->vaderSentiment) (3.6)

Requirement already satisfied: urllib3<3,>=1.21.1 in

/usr/local/lib/python3.10/dist-packages (from requests->vaderSentiment) (2.0.7)

Requirement already satisfied: certifi>=2017.4.17 in

/usr/local/lib/python3.10/dist-packages (from requests->vaderSentiment) (2023.11.17)

Installing collected packages: vaderSentiment

Successfully installed vaderSentiment-3.3.2

3 Dashboard Creation and Sentiment Analysis Section with Vader

```
[ ]: # Initialize Sentiment Analyzer
analyzer = SentimentIntensityAnalyzer()

comment_to_theme = {
    "problem-solving": ["technical_skills", "innovation"],
    "communication": ["communication"],
    "team player": ["teamwork", "leadership"],
    "technical expertise": ["technical_skills"],
    "leadership": ["leadership", "work_ethic"],
    "initiative": ["innovation", "work_ethic"],
    "creative": ["innovation"],
    "time management": ["time_management"],
    "performance targets": ["work_ethic", "technical_skills"],
    "client management": ["client_management"],
    "work ethic": ["work_ethic"],
    "new technologies": ["adaptability", "technical_skills"],
    "collaborative projects": ["teamwork", "communication"],
    "learning and growth": ["learning_growth"],
    "new challenges": ["adaptability", "innovation"],
    "under pressure": ["resilience", "work_ethic"],
    "detailed and accurate work": ["technical_skills"],
    "teamwork opportunities": ["teamwork"],
    "innovation and improvement": ["innovation", "adaptability"],
    "mentoring and coaching": ["leadership", "learning_growth"]
}

# Function to calculate sentiment score
def calculate_sentiment(text):
    return analyzer.polarity_scores(text)['compound']

# Function to process and load the Excel file
def load_excel(file_path_or_buffer):
    df = pd.read_excel(file_path_or_buffer)

    # Calculating sentiment scores
    df['Manager Comment Score'] = df['Manager Comment'].
    ↪apply(calculate_sentiment)
    df['Employee Review Score'] = df['Employee Performance Review'].
    ↪apply(calculate_sentiment)

    # Calculating Benchmark score
```

```

    df['Benchmark'] = 0.7 * df['Manager Comment Score'] + 0.3 * df['Employee_
↳Review Score']

    return df

# Function to display rankings
def display_rankings(df, position):
    if 'Position' not in df.columns:
        print("DataFrame not loaded or 'Position' column missing.")
        return
    subset = df[df['Position'] == position]
    ranked = subset.sort_values(by='Benchmark', ascending=False)
    globals()['current_view_df'] = ranked # Keep a reference to the filtered_
↳DataFrame for exporting
    with ranking_output_area:
        clear_output()
        display(ranked[['Employee Name', 'Department', 'Benchmark']])

# Function to display employee details with improved formatting using HTML
def display_employee_details(employee_name, keyword=None):
    if 'df' in globals():
        employee_data = globals()['df'][globals()['df']['Employee Name'] ==_
↳employee_name]
        if not employee_data.empty:
            # Start building the HTML string
            html_output = "<div style='font-family: Arial; padding: 10px;'"
            for column, value in employee_data.iloc[0].items():
                # Highlight the keyword if it's in the manager's comment
                if keyword and column == 'Manager Comment':
                    value = value.replace(keyword, f"<mark_
↳style='background-color: yellow;'{keyword}</mark>")
                    html_output += f"<strong>{column}</strong> <span style='color:_
↳#555;'{value}</span><br><br>"
                    html_output += "</div>"

            with details_output_area:
                clear_output()
                display(HTML(html_output))

# Function to aggregate data and create visualizations
def aggregate_department_data(df):
    return df.groupby('Department')['Benchmark'].mean().
↳sort_values(ascending=False)

# Function to find themes in a text based on keywords
def find_themes_in_text(text, themes):

```

```

found_themes = []
for theme, keywords in themes.items():
    if any(re.search(keyword, text, re.IGNORECASE) for keyword in keywords):
        found_themes.append(theme)
return found_themes

# Function to analyze reviews and count theme occurrences in negative reviews
def analyze_reviews_for_negative_themes(df, themes):
    negative_theme_counter = Counter()

    for index, row in df.iterrows():
        # Check if the review is negative
        if row['Manager Comment Score'] < 0 or row['Employee Review Score'] < 0:
            # Find themes in the manager's comment and employee review
            themes_in_comment = find_themes_in_text(row['Manager Comment'], themes)
            themes_in_review = find_themes_in_text(row['Employee Performance Review'], themes)

            # Update the counter for each found theme
            negative_theme_counter.update(themes_in_comment)
            negative_theme_counter.update(themes_in_review)

    return negative_theme_counter

# Function to find the most problematic theme for a department
def find_problematic_theme_for_department(df, department, themes):
    department_df = df[df['Department'] == department]
    negative_theme_counter = analyze_reviews_for_negative_themes(department_df, themes)

    if negative_theme_counter:
        most_common_theme, count = negative_theme_counter.most_common(1)[0]
        theme_description = ' and '.join(themes[most_common_theme])
        return f"Lacks {theme_description}", count
    return "No prominent negative theme", 0

import matplotlib.pyplot as plt
import numpy as np

def display_department_dashboard(department_data, df, themes):
    with dashboard_output_area:
        clear_output()
        plt.figure(figsize=(10, 6))

        # Identify the lowest performing department
        lowest_dept = department_data.idxmin()

```

```

        # Find the most problematic theme in this department
        problematic_theme_desc, count = _
    ↪ find_problematic_theme_for_department(df, lowest_dept, themes)

    # Coloring for bars
    colors = ['skyblue' if dept != lowest_dept else 'salmon' for dept in _
    ↪ department_data.index]

    # Creating the bar plot
    bars = plt.bar(department_data.index, department_data.values, _
    ↪ color=colors)

    # Highlighting the lowest department
    for bar, dept in zip(bars, department_data.index):
        if dept == lowest_dept:
            bar.set_edgecolor('red')
            bar.set_linewidth(1.5)
            plt.text(bar.get_x() + bar.get_width() / 2.0, bar.get_height(),
                     f'{problematic_theme_desc} ({count})',
                     ha='center', va='bottom', color='red', fontsize=9)

    # Adding titles and labels
    plt.title('Average Benchmark Score by Department')
    plt.xlabel('Department')
    plt.ylabel('Average Benchmark Score')
    plt.xticks(rotation=45)

    # Show the plot
    plt.show()

# UI Elements
file_upload = widgets.FileUpload(description='Upload File', accept='.xlsx', _
    ↪ multiple=False)
position_picker = widgets.Dropdown(description='Position:', disabled=True)
ranking_output_area = widgets.Output()
details_output_area = widgets.Output()
dashboard_output_area = widgets.Output()
employee_search_box = widgets.Text(placeholder='Type to search...')
employee_list = widgets.Select()
export_filtered_button = widgets.Button(description="Export Filtered Data")

# Function to Export Filtered Data
def export_filtered_data(b):
    if 'current_view_df' in globals():
        try:

```

```

        # Export to Excel
        current_view_df = globals()['current_view_df']
        output_file_path = 'filtered_data.xlsx' # Define a file name
        current_view_df.to_excel(output_file_path)
        print("Filtered data prepared for download.")
        files.download(output_file_path) # Trigger download
    except Exception as e:
        print(f"An error occurred while exporting: {e}")
    else:
        print("No filtered data to export.")

# Bind the export function to the new export button
export_filtered_button.on_click(export_filtered_data)

# Event Handlers
def on_file_upload_change(change):
    # Clear previous data
    with ranking_output_area:
        clear_output(wait=True)
    with details_output_area:
        clear_output(wait=True)
    with dashboard_output_area:
        clear_output(wait=True)
    uploaded_filename = next(iter(file_upload.value))
    content = file_upload.value[uploaded_filename]['content']
    df = load_excel(io.BytesIO(content))
    if 'Position' in df.columns and 'Department' in df.columns:
        position_picker.options = df['Position'].unique()
        position_picker.disabled = False
        department_data = aggregate_department_data(df)
        display_department_dashboard(department_data, df, comment_to_theme)
    else:
        print("Error: Required columns not found in the uploaded file.")
    globals()['df'] = df

def on_position_picker_change(change):
    if change.new and 'df' in globals():
        display_rankings(globals()['df'], change.new)

def on_employee_search_change(change):
    search_term = change.new
    position = position_picker.value
    if position and 'df' in globals():
        filtered_employees = filter_employees(search_term, position)
        employee_list.options = filtered_employees

def on_employee_selected(change):

```

```

    if change.new and 'df' in globals():
        display_employee_details(change.new)

def filter_employees(search_term, position):
    if 'df' in globals():
        position_df = globals()['df']
        all_employees = position_df[position_df['Position'] ==
        ↪position]['Employee Name'].unique()
        filtered_employees = [e for e in all_employees if search_term.lower()
        ↪in e.lower()]
        return filtered_employees
    return []

# UI Elements for searching based on manager comments
comment_search_box = widgets.Text(placeholder='Enter keyword from manager
    ↪comments...')
comment_search_results = widgets.Output()

# Function to filter employees based on a keyword in manager comments and
    ↪position
def filter_employees_by_comment_keyword(keyword, position):
    if 'df' in globals() and keyword and position:
        df = globals()['df']
        # Filter employees based on keyword and position
        filtered_employees = df[(df['Position'] == position) & (df['Manager
        ↪Comment'].str.contains(keyword, case=False, na=False))]

        # Sort by 'Benchmark' score in descending order
        sorted_employees = filtered_employees.sort_values(by='Benchmark',
        ↪ascending=False)

        # Get the top 4 positive and 4 worst employees based on Benchmark
        top_positive = sorted_employees.head(4)
        worst = sorted_employees.tail(4)

        # Combine both lists and return unique names
        combined_employees = pd.concat([top_positive, worst])['Employee Name'].
        ↪unique()
        return combined_employees
    return []

# Update employee list based on comment keyword search and position
def on_comment_search_change(change):
    keyword = change.new
    position = position_picker.value
    if position:

```

```

        sorted_employees = filter_employees_by_comment_keyword(keyword,
↪position)
        with comment_search_results:
            clear_output()
            if sorted_employees.size > 0:
                for employee in sorted_employees:
                    print(employee)
                    # Update the display function to include the keyword for
↪highlighting
                    display_employee_details(employee, keyword=keyword)
            else:
                print("No employees found with the specified keyword in manager
↪comments for the selected position.")

comment_search_box.observe(on_comment_search_change, 'value')

# Assigning Event Handlers
file_upload.observe(on_file_upload_change, names='value')
position_picker.observe(on_position_picker_change, names='value')
employee_search_box.observe(on_employee_search_change, 'value')
employee_list.observe(on_employee_selected, names='value')

display(widgets.HBox([
    widgets.VBox([
        file_upload,
        position_picker,
        widgets.HBox([employee_search_box, comment_search_box]), # Search
↪boxes side by side
        widgets.HBox([export_filtered_button]), # Place the export button on a
↪new line
        employee_list,
        comment_search_results, # Component for displaying search results
↪based on manager comments
    ]),
    widgets.VBox([
        ranking_output_area,
        details_output_area,
        dashboard_output_area
    ])
]))

```

```

HBox(children=(VBox(children=(FileUpload(value={}, accept='.xlsx',
↪description='Upload File'), Dropdown(descri...

```

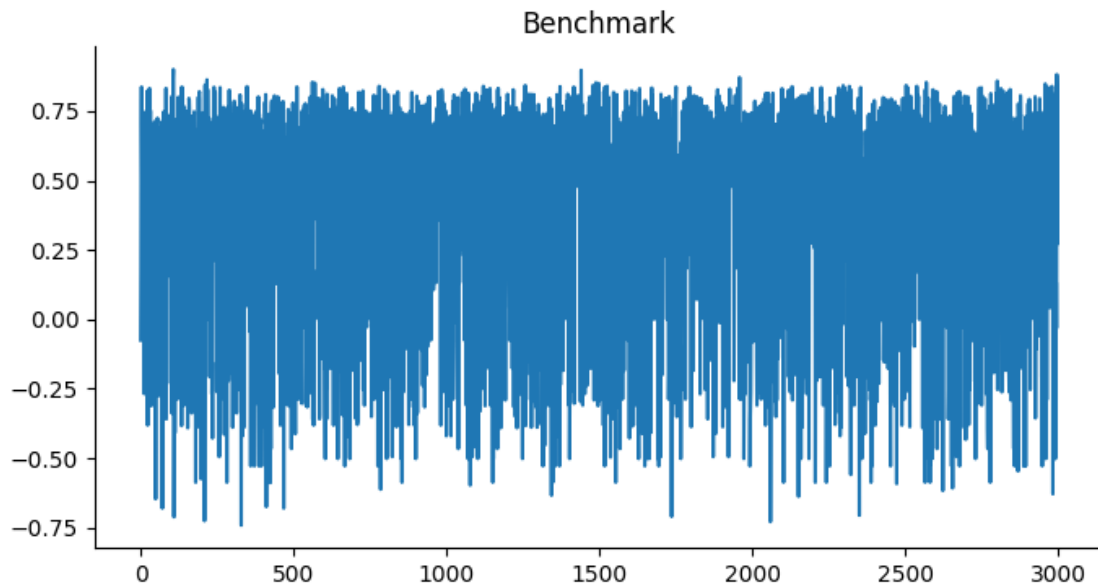
Filtered data prepared for download.

<IPython.core.display.Javascript object>

<IPython.core.display.Javascript object>

4 Chart Section

```
[ ]: from matplotlib import pyplot as plt
df['Benchmark'].plot(kind='line', figsize=(8, 4), title='Benchmark')
plt.gca().spines[['top', 'right']].set_visible(False)
```

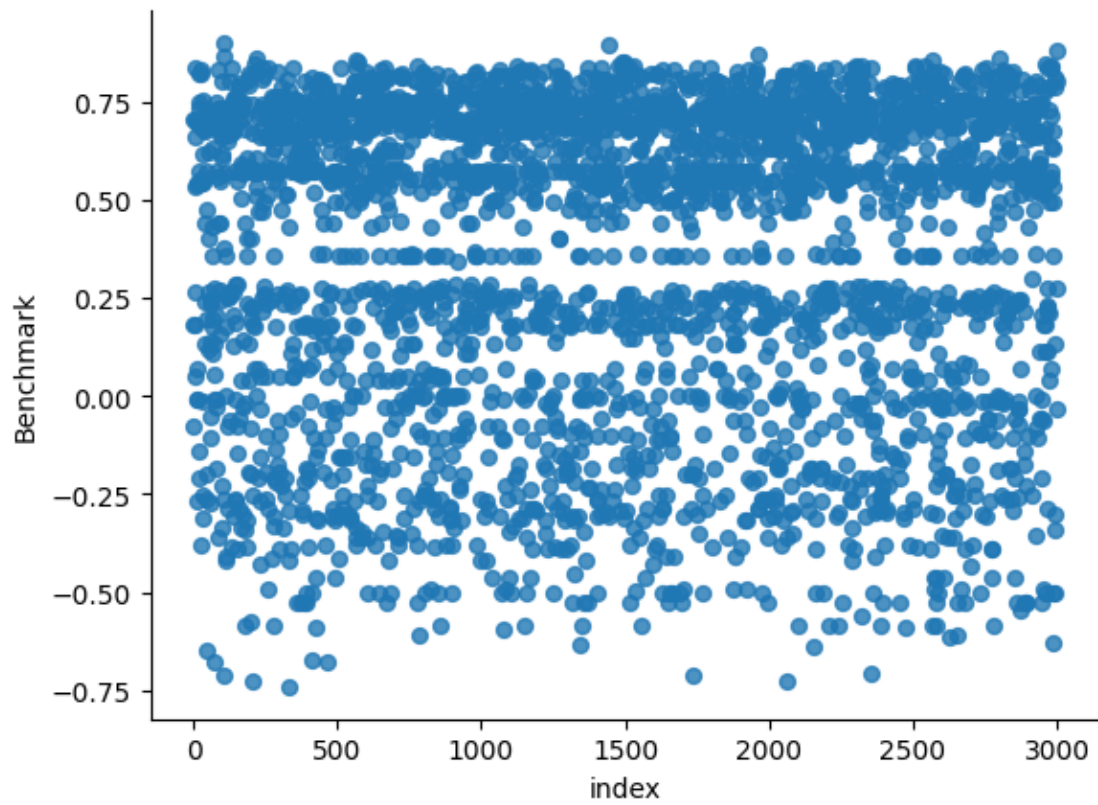


```
[ ]: from matplotlib import pyplot as plt

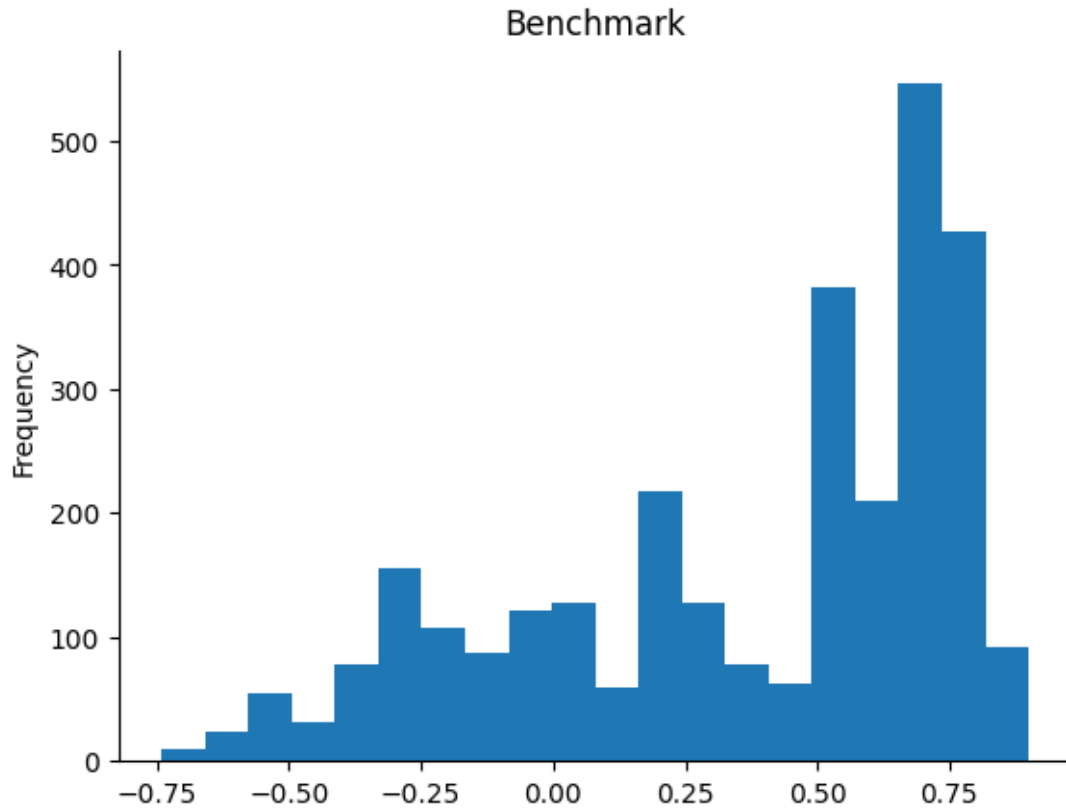
# Reset the index to turn it into a regular column
df_reset = df.reset_index()

# Plot using the new 'index' column
df_reset.plot(kind='scatter', x='index', y='Benchmark', s=32, alpha=0.8)

# Adjusting the appearance of the plot
plt.gca().spines[['top', 'right']].set_visible(False)
plt.show() # Display the plot
```



```
[ ]: from matplotlib import pyplot as plt
df['Benchmark'].plot(kind='hist', bins=20, title='Benchmark')
plt.gca().spines[['top', 'right']].set_visible(False)
```



```
[ ]: !pip install xlswriter
!pip install reportlab
from google.colab import drive
drive.mount('/content/drive')

import io
import os
import tempfile
import pandas as pd
import matplotlib.pyplot as plt
import ipywidgets
from ipywidgets import widgets, Button
from IPython.display import FileLink, display, clear_output
import numpy as np
import seaborn as sns
import nltk
from reportlab.lib.pagesizes import letter
from reportlab.pdfgen import canvas
from reportlab.lib.utils import ImageReader
from reportlab.platypus import SimpleDocTemplate, Image, Spacer, Paragraph, Image
from reportlab.lib.styles import getSampleStyleSheet
```

```

try:
    from nltk.sentiment import SentimentIntensityAnalyzer
except ImportError:

    from nltk.sentiment import SentimentIntensityAnalyzer
nltk.download('vader_lexicon')
# Initialize Sentiment Analyzer
analyzer = SentimentIntensityAnalyzer()

```

Requirement already satisfied: xlswriter in /usr/local/lib/python3.10/dist-packages (3.1.9)

Requirement already satisfied: reportlab in /usr/local/lib/python3.10/dist-packages (4.0.8)

Requirement already satisfied: pillow>=9.0.0 in /usr/local/lib/python3.10/dist-packages (from reportlab) (9.4.0)

Requirement already satisfied: chardet in /usr/local/lib/python3.10/dist-packages (from reportlab) (5.2.0)

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).

[nltk_data] Downloading package vader_lexicon to /root/nltk_data...

[nltk_data] Package vader_lexicon is already up-to-date!

5 Employee Insights by different Metrics and Final Sentiment Scores Analysis

```

[ ]: modified_df = None

# Function to calculate sentiment score
def calculate_sentiment(text):
    return analyzer.polarity_scores(text)['compound']
# Function to display the plot on the screen
def display_overall_performance_distribution(df):
    fig, ax = plt.subplots(figsize=(10, 6))
    sns.countplot(data=df, x='Performance Classification', order=['High Performer', 'Normal Performer', 'Low Performer'], palette='coolwarm', ax=ax)
    ax.set_title('Overall Performance Classification Distribution')
    ax.set_ylabel('Number of Employees')
    ax.set_xlabel('Performance Classification')
    total = len(df)
    for p in ax.patches:
        count = p.get_height()
        percentage = 100 * count / total
        text = f'{count}\n({percentage:.1f}%)'
        x = p.get_x() + p.get_width() / 2
        y = p.get_height()
        ax.annotate(text, (x, y), ha='center', va='bottom')

```

```

plt.tight_layout()
plt.show()
def plot_overall_performance_distribution(df):
    # Create the figure and axis for the plot
    fig, ax = plt.subplots(figsize=(10, 6))

    # Create the count plot
    sns.countplot(data=df, x='Performance Classification', order=['High Performer', 'Normal Performer', 'Low Performer'], palette='coolwarm', ax=ax)

    # Title and labels
    ax.set_title('Overall Performance Classification Distribution')
    ax.set_ylabel('Number of Employees')
    ax.set_xlabel('Performance Classification')

    # Calculate the total number for annotations
    total = len(df)

    # Annotate the bars with the count and percentage
    for p in ax.patches:
        count = p.get_height() # The count for each category
        percentage = 100 * count / total # The percentage of each category
        text = f'{count}\n({percentage:.1f}%)' # The annotation text
        x = p.get_x() + p.get_width() / 2 # x position for the text
        y = p.get_height() # y position for the text
        ax.annotate(text, (x, y), ha='center', va='bottom')

    # Save the plot to a BytesIO buffer
    img_data = io.BytesIO()
    fig.savefig(img_data, format='png')
    img_data.seek(0)
    plt.close(fig)

    return img_data
# Function to save the plot
def classify_performance(row):
    if row['Benchmark'] > 0.67:
        return 'High Performer'
    elif row['Benchmark'] >= 0.2:
        return 'Normal Performer'
    else:
        return 'Low Performer'
def save_plot(figure):
    img_data = io.BytesIO()
    figure.savefig(img_data, format='png')
    img_data.seek(0) # Go to the start of the BytesIO buffer
    plt.close(figure)

```

```

    return img_data
# Function to create the PDF report and save it to Google Drive
def create_pdf_report(plot_buffers, filename="Performance_Report.pdf"):
    local_path = f'/content/{filename}'
    doc = SimpleDocTemplate(local_path, pagesize=letter)
    story = []
    styles = getSampleStyleSheet()

    story.append(Paragraph("Performance Analysis Report", styles['Title']))

    for plot_buffer in plot_buffers:
        story.append(Image(plot_buffer, width=400, height=200)) # Adjust size
        # as needed
        story.append(Spacer(1, 12))

    doc.build(story)
    print(f"PDF created at {local_path}")

    # Move the file to Google Drive folder
    drive_path = '/content/drive/MyDrive/MasterNeapolis/'
    !mv {local_path} {drive_path}
    print(f"Report saved to Google Drive at: {drive_path}{filename}")
    return f"{drive_path}{filename}"

# Create a file upload widget
upload_button = widgets.FileUpload(
    accept='.xlsx',
    multiple=False,
    description='Upload Excel File'
)

# Function to download the modified DataFrame as Excel
def download_excel():
    if modified_df is not None:
        # Define a local file path
        local_path = 'ModifiedData.xlsx'

        # Save the DataFrame to an Excel file
        modified_df.to_excel(local_path, index=False)

        # Move the file to Google Drive folder (adjust the path as needed)
        drive_path = '/content/drive/MyDrive/MasterNeapolis/'
        !mv {local_path} {drive_path}

        print(f"File saved to Google Drive at: {drive_path}")

```

```

# Function to process the uploaded file and create plots
def process_and_plot(change):
    uploaded_file = change['new']
    global modified_df
    plot_buffers = [] # Store plot images in memory
    if uploaded_file:
        clear_output(wait=True) # Clear the previous output
        display(upload_button, download_button) # Display the upload button
        ↪again
        # Read the Excel file
        excel_file = list(uploaded_file.values())[0]
        df = pd.read_excel(io.BytesIO(excel_file['content']))

        # Check if 'Manager Comment Score' column exists, if not calculate it
        if 'Manager Comment Score' not in df.columns:
            df['Manager Comment Score'] = df['Manager Comment'].
            ↪apply(calculate_sentiment)
            if 'Employee Performance Review Score' not in df.columns:
                df['Employee Performance Review Score'] = df['Employee Performance
                ↪Review'].apply(calculate_sentiment)

            # Calculate the 'Benchmark' score
            df['Benchmark'] = 0.7 * df['Manager Comment Score'] + 0.3 *
            ↪df['Employee Performance Review Score']
            # Classification step
            df['Performance Classification'] = df.apply(classify_performance,
            ↪axis=1)
            modified_df = df

            # Performance by Ethnicity with Highlighted Low Performer
            fig=plt.figure(figsize=(12, 6))
            ethnicity_performance = df.groupby('Ethnicity')['Benchmark'].mean().
            ↪sort_values()
            sns.barplot(x=ethnicity_performance.values, y=ethnicity_performance.
            ↪index, palette="viridis")
            lowest_ethnicity = ethnicity_performance.idxmin()
            plt.barh(lowest_ethnicity, ethnicity_performance.min(), color='red')
            plt.xlabel('Average Benchmark Score')
            plt.ylabel('Ethnicity')
            plt.title('Performance by Ethnicity')
            plt.grid(axis='x')
            plt.show()
            plot_buffers.append(save_plot(fig))
            #save_plot(plt.gcf(), 'ethnicity_performance')

            # Create and append the new performance distribution plot

```

```

# Display the plot on the screen
display_overall_performance_distribution(df)
fig = plot_overall_performance_distribution(df)
plot_buffers.append(fig)

# Performance by Age Group with Highlighted Low Performer
fig=plt.figure(figsize=(12, 6))
df['Age Group'] = pd.cut(df['Age'], bins=[20, 30, 40, 50, 60, 70],
↳labels=['21-30', '31-40', '41-50', '51-60', '61-70'])
age_performance = df.groupby('Age Group')['Benchmark'].mean().
↳sort_values()
sns.barplot(x=age_performance.index, y=age_performance.values,
↳palette="mako")
lowest_age_group = age_performance.idxmin()
plt.bar(lowest_age_group, age_performance.min(), color='red')
plt.ylabel('Average Benchmark Score')
plt.xlabel('Age Group')
plt.title('Performance by Age Group')
plt.grid(axis='y')
plt.show()
#save_plot(plt.gcf(), 'performance_age_Group')
plot_buffers.append(save_plot(fig))

# Chart for Count of Employees by Ethnicity
fig=plt.figure(figsize=(12, 6))
ethnicity_counts = df['Ethnicity'].value_counts()
lowest_count_ethnicity = ethnicity_counts.idxmin()
colors = ['red' if eth == lowest_count_ethnicity else 'blue' for eth in
↳ethnicity_counts.index]

bars = plt.bar(ethnicity_counts.index, ethnicity_counts.values,
↳color=colors)

# Add the actual count on each bar
for bar in bars:
    yval = bar.get_height()
    plt.text(bar.get_x() + bar.get_width() / 2, yval, int(yval),
↳va='bottom', ha='center')

plt.xlabel('Ethnicity')
plt.ylabel('Number of Employees')
plt.title('Count of Employees by Ethnicity')
plt.xticks(rotation=45)
plt.show()

```



```

#save_plot(plt.gcf(), 'employees_ethnicity')
plot_buffers.append(save_plot(fig))

# Plotting Position Level Performance
fig=plt.figure(figsize=(12, 6))
position_performance = df.groupby('Position')['Benchmark'].mean().
↳sort_values()
sns.barplot(x=position_performance.values, y=position_performance.
↳index, palette="rocket")
plt.xlabel('Average Benchmark Score')
plt.ylabel('Position')
plt.title('Performance by Job Position')
plt.grid(axis='x')
plt.show()
#save_plot(plt.gcf(), 'Position_Performance')
plot_buffers.append(save_plot(fig))

# Correlation Heatmap Age vs Experience Years
df['Age'] = pd.to_numeric(df['Age'], errors='coerce')
df['Years of Experience'] = pd.to_numeric(df['Years of Experience'],
↳errors='coerce')

# Selecting relevant columns for correlation
corr_columns = df[['Age', 'Years of Experience', 'Manager Comment',
↳Score', 'Employee Performance Review Score', 'Benchmark']]
corr = corr_columns.corr()

# Mask for the upper triangle
mask = np.triu(np.ones_like(corr, dtype=bool))

# Creating the heatmap
fig=plt.figure(figsize=(12, 10))
sns.heatmap(corr, annot=True, cmap='coolwarm', fmt='.2f', mask=mask)
plt.title('Correlation Heatmap', fontsize=16)

# Annotation for Age vs. Years of Experience
plt.text(1.5, 0.5, "Older employees\nusually have more\nwork_
↳experience",
        ha='center', va='center', fontsize=12, color='black',
↳bbox=dict(facecolor='white', alpha=0.5))

plt.xticks(fontsize=12)
plt.yticks(fontsize=12)
plt.show()
#save_plot(plt.gcf(), 'Heatmap')
plot_buffers.append(save_plot(fig))

```

```

modified_df = df
report_filename = create_pdf_report(plot_buffers)
print(f"Report generated: {report_filename}")

# Create a download button
download_button = widgets.Button(description='Download Excel File')
download_button.on_click(lambda _: display(download_excel()))
# Observe changes in the file upload widget
upload_button.observe(process_and_plot, names='value')

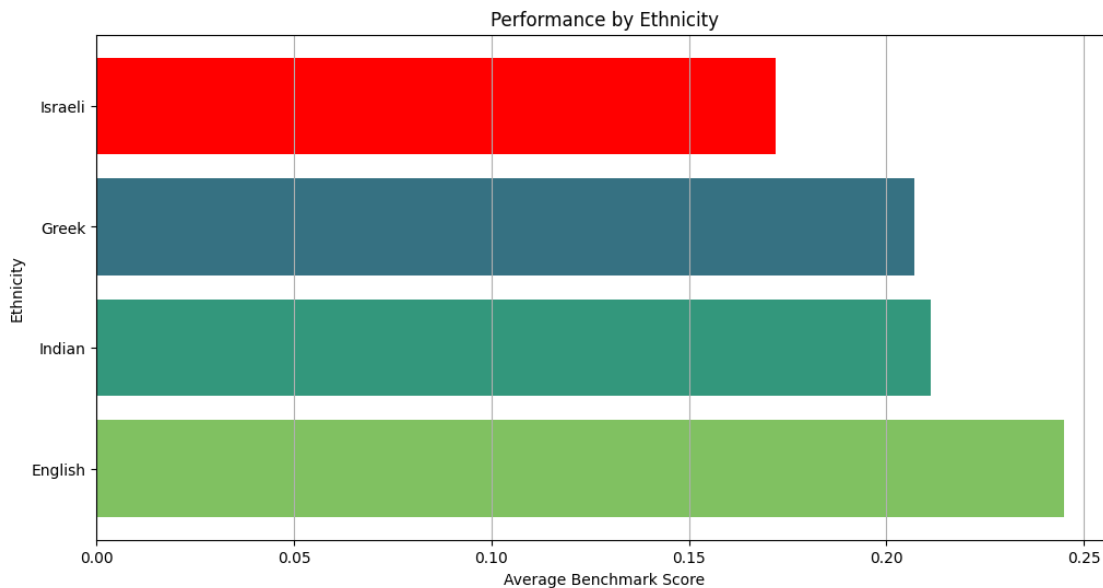
# Display the upload button and download button
display(upload_button, download_button)

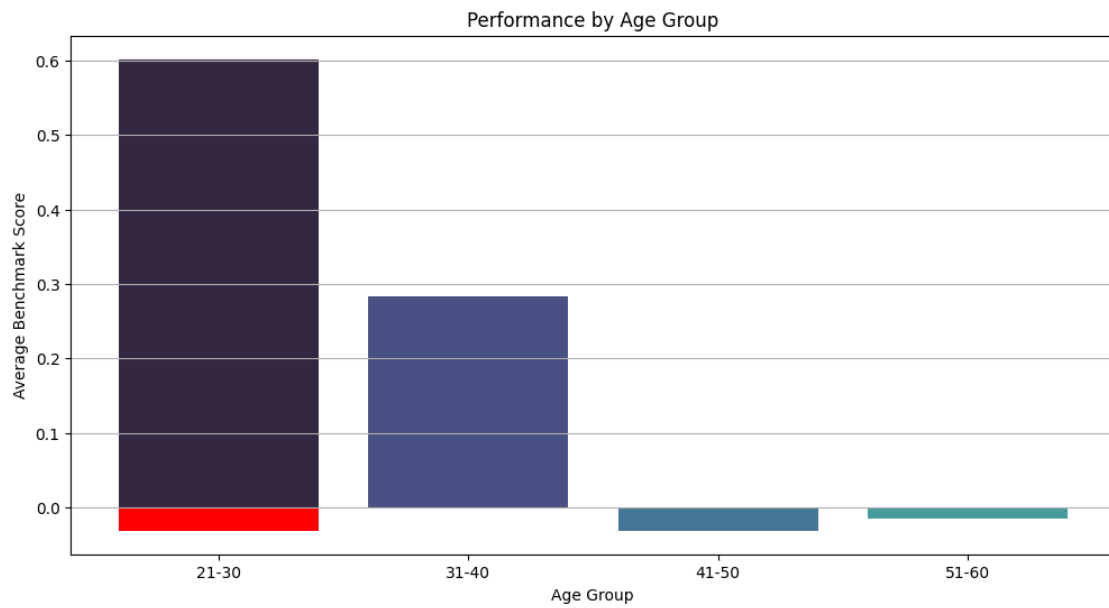
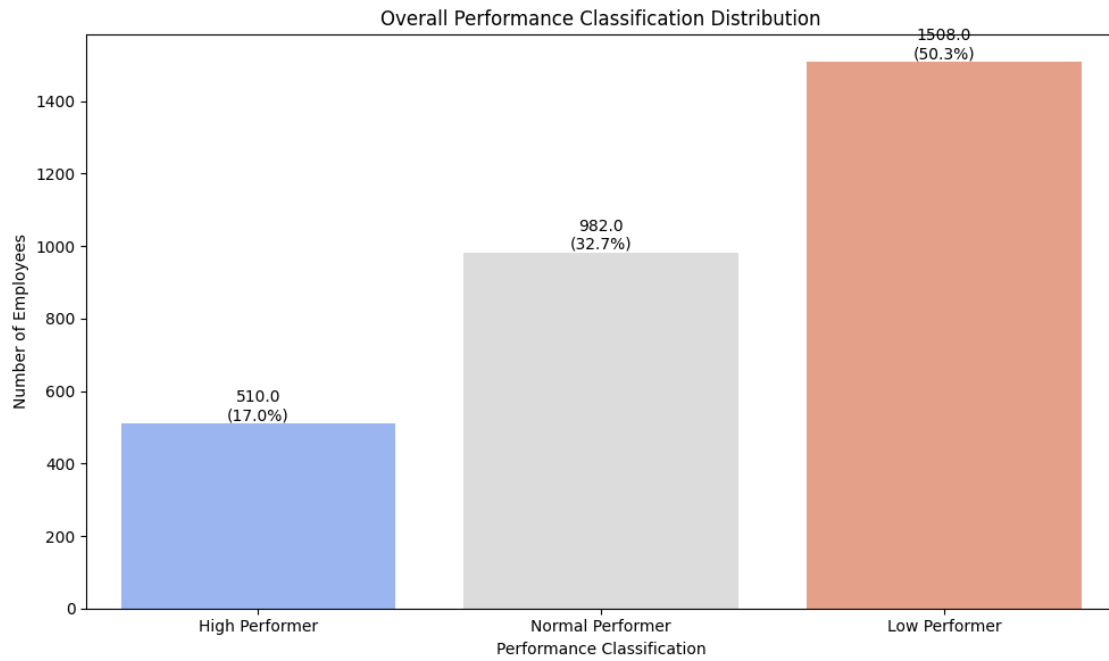
# Observe changes in the file upload widget
#upload_button.observe(process_and_plot, names='value')

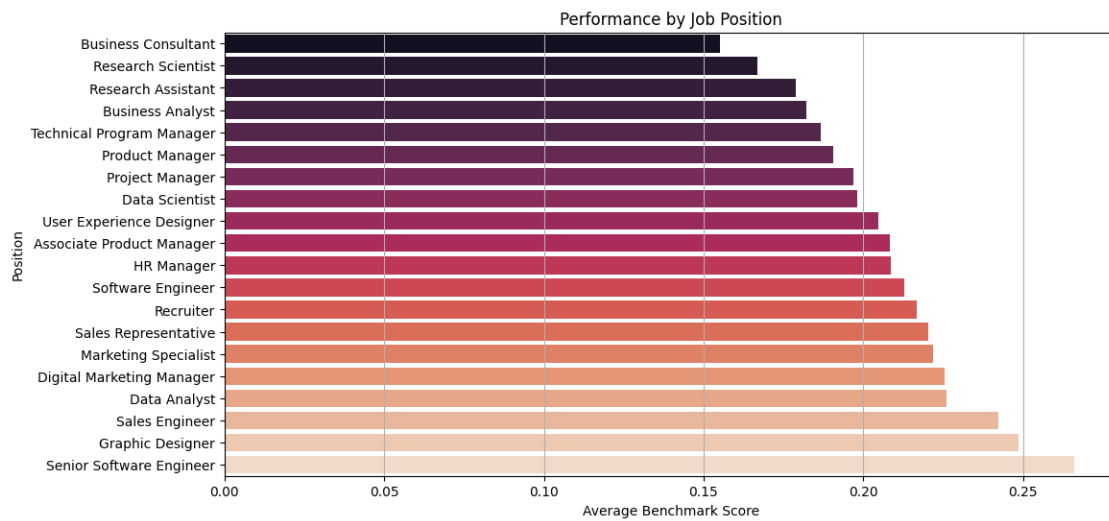
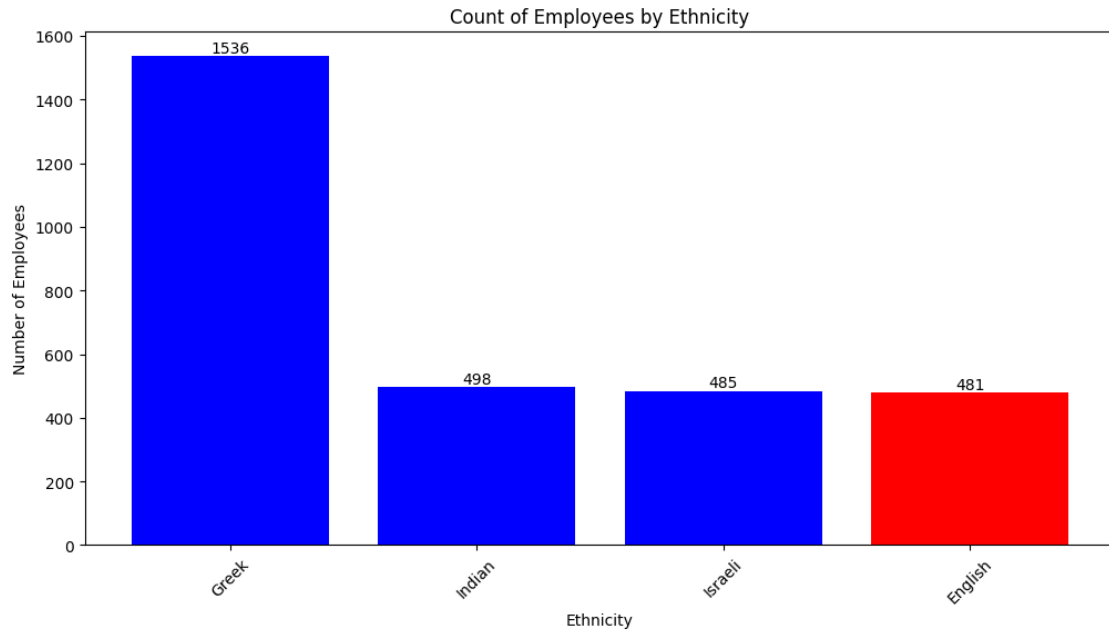
FileUpload(value={'biasData4.xlsx': {'metadata': {'name': 'biasData4.xlsx',
↳ 'type': 'application/vnd.openxmlfo...

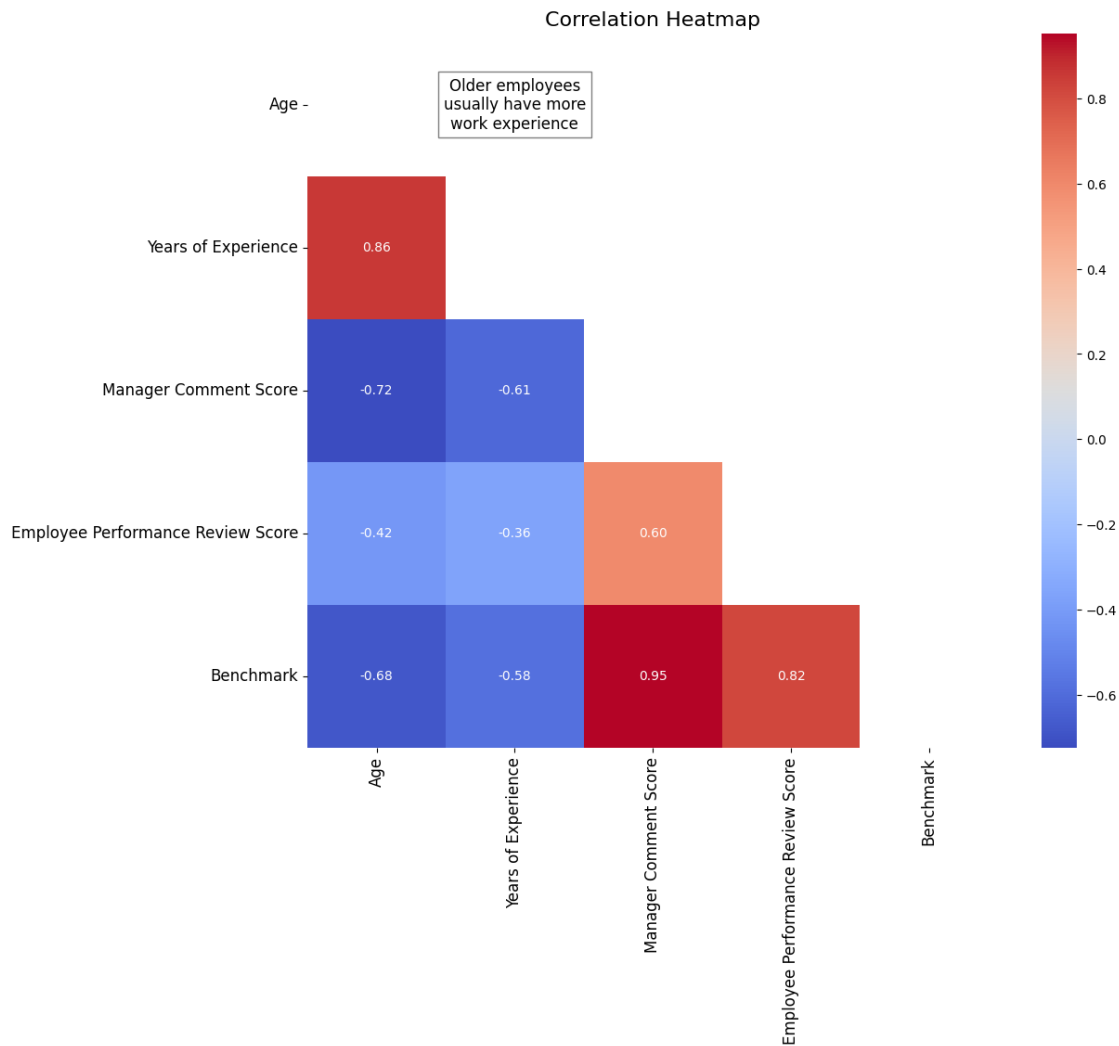
Button(description='Download Excel File', style=ButtonStyle())

```









PDF created at /content/Performance_Report.pdf

Report saved to Google Drive at:

/content/drive/MyDrive/MasterNeapolis/Performance_Report.pdf

Report generated: /content/drive/MyDrive/MasterNeapolis/Performance_Report.pdf

File saved to Google Drive at: /content/drive/MyDrive/MasterNeapolis/

None

6 Data Preprocessing, tokenization and preparation for Model Training

```
[1]: import pandas as pd
from sklearn.model_selection import train_test_split
import re
```

```

# Load the dataset
file_path = '/content/HR_Recommendations.xlsx' # Update the path if needed
df = pd.read_excel(file_path)

# Select relevant columns
df = df[['Manager Comment', 'HR_Insight']]

# Data Cleaning and Preprocessing
def clean_text(text):
    text = str(text).lower() # Convert to lowercase
    text = re.sub(r'\s+', ' ', text) # Remove extra spaces
    text = re.sub(r'[\w\s]', '', text) # Remove punctuation
    return text

df['Manager Comment'] = df['Manager Comment'].apply(clean_text)
df['HR_Insight'] = df['HR_Insight'].apply(clean_text)

# Splitting the data
train, test = train_test_split(df, test_size=0.3, random_state=42)
val, test = train_test_split(test, test_size=0.5, random_state=42)

# Saving the data
train.to_csv('/content/hr_train.csv', index=False)
val.to_csv('/content/hr_val.csv', index=False)
test.to_csv('/content/hr_test.csv', index=False)

print("Data preparation complete. Files saved as hr_train.csv, hr_val.csv, and_
↳hr_test.csv")

```

Data preparation complete. Files saved as hr_train.csv, hr_val.csv, and hr_test.csv

```
[ ]: !ls "/content/drive/MyDrive/MasterNeapolis/Model/"
```

ls: cannot access '/content/drive/MyDrive/MasterNeapolis/Model/': No such file or directory

7 Model Import GPT2

```

[2]: !pip install transformers
!pip install accelerate -U
!pip install transformers -U

from transformers import GPT2LMHeadModel, GPT2Tokenizer

tokenizer = GPT2Tokenizer.from_pretrained("gpt2")
model = GPT2LMHeadModel.from_pretrained("gpt2")

```

Requirement already satisfied: transformers in /usr/local/lib/python3.10/dist-packages (4.35.2)
Requirement already satisfied: filelock in /usr/local/lib/python3.10/dist-packages (from transformers) (3.13.1)
Requirement already satisfied: huggingface-hub<1.0,>=0.16.4 in /usr/local/lib/python3.10/dist-packages (from transformers) (0.19.4)
Requirement already satisfied: numpy>=1.17 in /usr/local/lib/python3.10/dist-packages (from transformers) (1.23.5)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.10/dist-packages (from transformers) (23.2)
Requirement already satisfied: pyyaml>=5.1 in /usr/local/lib/python3.10/dist-packages (from transformers) (6.0.1)
Requirement already satisfied: regex!=2019.12.17 in /usr/local/lib/python3.10/dist-packages (from transformers) (2023.6.3)
Requirement already satisfied: requests in /usr/local/lib/python3.10/dist-packages (from transformers) (2.31.0)
Requirement already satisfied: tokenizers<0.19,>=0.14 in /usr/local/lib/python3.10/dist-packages (from transformers) (0.15.0)
Requirement already satisfied: safetensors>=0.3.1 in /usr/local/lib/python3.10/dist-packages (from transformers) (0.4.1)
Requirement already satisfied: tqdm>=4.27 in /usr/local/lib/python3.10/dist-packages (from transformers) (4.66.1)
Requirement already satisfied: fsspec>=2023.5.0 in /usr/local/lib/python3.10/dist-packages (from huggingface-hub<1.0,>=0.16.4->transformers) (2023.6.0)
Requirement already satisfied: typing-extensions>=3.7.4.3 in /usr/local/lib/python3.10/dist-packages (from huggingface-hub<1.0,>=0.16.4->transformers) (4.5.0)
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.10/dist-packages (from requests->transformers) (3.3.2)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.10/dist-packages (from requests->transformers) (3.6)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.10/dist-packages (from requests->transformers) (2.0.7)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.10/dist-packages (from requests->transformers) (2023.11.17)

Collecting accelerate

Downloading accelerate-0.25.0-py3-none-any.whl (265 kB)

265.7/265.7

kB 5.5 MB/s eta 0:00:00

Requirement already satisfied: numpy>=1.17 in /usr/local/lib/python3.10/dist-packages (from accelerate) (1.23.5)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.10/dist-packages (from accelerate) (23.2)
Requirement already satisfied: psutil in /usr/local/lib/python3.10/dist-packages (from accelerate) (5.9.5)

Requirement already satisfied: pyyaml in /usr/local/lib/python3.10/dist-packages (from accelerate) (6.0.1)

Requirement already satisfied: torch>=1.10.0 in /usr/local/lib/python3.10/dist-packages (from accelerate) (2.1.0+cu121)

Requirement already satisfied: huggingface-hub in /usr/local/lib/python3.10/dist-packages (from accelerate) (0.19.4)

Requirement already satisfied: safetensors>=0.3.1 in /usr/local/lib/python3.10/dist-packages (from accelerate) (0.4.1)

Requirement already satisfied: filelock in /usr/local/lib/python3.10/dist-packages (from torch>=1.10.0->accelerate) (3.13.1)

Requirement already satisfied: typing-extensions in /usr/local/lib/python3.10/dist-packages (from torch>=1.10.0->accelerate) (4.5.0)

Requirement already satisfied: sympy in /usr/local/lib/python3.10/dist-packages (from torch>=1.10.0->accelerate) (1.12)

Requirement already satisfied: networkx in /usr/local/lib/python3.10/dist-packages (from torch>=1.10.0->accelerate) (3.2.1)

Requirement already satisfied: Jinja2 in /usr/local/lib/python3.10/dist-packages (from torch>=1.10.0->accelerate) (3.1.2)

Requirement already satisfied: fsspec in /usr/local/lib/python3.10/dist-packages (from torch>=1.10.0->accelerate) (2023.6.0)

Requirement already satisfied: triton==2.1.0 in /usr/local/lib/python3.10/dist-packages (from torch>=1.10.0->accelerate) (2.1.0)

Requirement already satisfied: requests in /usr/local/lib/python3.10/dist-packages (from huggingface-hub->accelerate) (2.31.0)

Requirement already satisfied: tqdm>=4.42.1 in /usr/local/lib/python3.10/dist-packages (from huggingface-hub->accelerate) (4.66.1)

Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.10/dist-packages (from Jinja2->torch>=1.10.0->accelerate) (2.1.3)

Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.10/dist-packages (from requests->huggingface-hub->accelerate) (3.3.2)

Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.10/dist-packages (from requests->huggingface-hub->accelerate) (3.6)

Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.10/dist-packages (from requests->huggingface-hub->accelerate) (2.0.7)

Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.10/dist-packages (from requests->huggingface-hub->accelerate) (2023.11.17)

Requirement already satisfied: mpmath>=0.19 in /usr/local/lib/python3.10/dist-packages (from sympy->torch>=1.10.0->accelerate) (1.3.0)

Installing collected packages: accelerate

Successfully installed accelerate-0.25.0

Requirement already satisfied: transformers in /usr/local/lib/python3.10/dist-packages (4.35.2)

Collecting transformers

Downloading transformers-4.36.2-py3-none-any.whl (8.2 MB)

8.2/8.2 MB

31.2 MB/s eta 0:00:00

Requirement already satisfied: filelock in /usr/local/lib/python3.10/dist-packages (from transformers) (3.13.1)
Requirement already satisfied: huggingface-hub<1.0,>=0.19.3 in /usr/local/lib/python3.10/dist-packages (from transformers) (0.19.4)
Requirement already satisfied: numpy>=1.17 in /usr/local/lib/python3.10/dist-packages (from transformers) (1.23.5)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.10/dist-packages (from transformers) (23.2)
Requirement already satisfied: pyyaml>=5.1 in /usr/local/lib/python3.10/dist-packages (from transformers) (6.0.1)
Requirement already satisfied: regex!=2019.12.17 in /usr/local/lib/python3.10/dist-packages (from transformers) (2023.6.3)
Requirement already satisfied: requests in /usr/local/lib/python3.10/dist-packages (from transformers) (2.31.0)
Requirement already satisfied: tokenizers<0.19,>=0.14 in /usr/local/lib/python3.10/dist-packages (from transformers) (0.15.0)
Requirement already satisfied: safetensors>=0.3.1 in /usr/local/lib/python3.10/dist-packages (from transformers) (0.4.1)
Requirement already satisfied: tqdm>=4.27 in /usr/local/lib/python3.10/dist-packages (from transformers) (4.66.1)
Requirement already satisfied: fsspec>=2023.5.0 in /usr/local/lib/python3.10/dist-packages (from huggingface-hub<1.0,>=0.19.3->transformers) (2023.6.0)
Requirement already satisfied: typing-extensions>=3.7.4.3 in /usr/local/lib/python3.10/dist-packages (from huggingface-hub<1.0,>=0.19.3->transformers) (4.5.0)
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.10/dist-packages (from requests->transformers) (3.3.2)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.10/dist-packages (from requests->transformers) (3.6)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.10/dist-packages (from requests->transformers) (2.0.7)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.10/dist-packages (from requests->transformers) (2023.11.17)

Installing collected packages: transformers

Attempting uninstall: transformers

Found existing installation: transformers 4.35.2

Uninstalling transformers-4.35.2:

Successfully uninstalled transformers-4.35.2

Successfully installed transformers-4.36.2

vocab.json: 0%| | 0.00/1.04M [00:00<?, ?B/s]

merges.txt: 0%| | 0.00/456k [00:00<?, ?B/s]

tokenizer.json: 0%| | 0.00/1.36M [00:00<?, ?B/s]

```
config.json: 0%|          | 0.00/665 [00:00<?, ?B/s]
model.safetensors: 0%|          | 0.00/548M [00:00<?, ?B/s]
generation_config.json: 0%|          | 0.00/124 [00:00<?, ?B/s]
```

8 Model Training and Evaluation-Fine-Tuning-Illustration of different phases of Metrics During Training

```
[6]: # Importing necessary libraries
from transformers import GPT2LMHeadModel, GPT2Tokenizer
import matplotlib.pyplot as plt
from transformers import TrainingArguments, Trainer, TrainerCallback
from torch.utils.data import Dataset
import pandas as pd
# Loading the model and tokenizer
tokenizer = GPT2Tokenizer.from_pretrained("gpt2")
model = GPT2LMHeadModel.from_pretrained("gpt2")

tokenizer.pad_token = tokenizer.eos_token # Set padding token
# Define a custom dataset class
class HRDataset(Dataset):
    def __init__(self, tokenizer, file_path, max_length=512):
        self.tokenizer = tokenizer
        self.data = pd.read_csv(file_path)
        self.max_length = max_length

    def __len__(self):
        return len(self.data)

    def __getitem__(self, idx):
        row = self.data.iloc[idx]
        manager_comment = row['Manager Comment']
        hr_insight = row['HR Insight']

        # Combine the input and output texts with a special token in between
        combined_text = manager_comment + tokenizer.eos_token + hr_insight
        encoding = self.tokenizer.encode_plus(combined_text, max_length=self.
        ↪max_length, truncation=True, padding="max_length", return_tensors="pt")

        # Use the input_ids as labels for training
        labels = encoding['input_ids']

    return {
        'input_ids': encoding['input_ids'].flatten(),
        'attention_mask': encoding['attention_mask'].flatten(),
        'labels': labels.flatten()
    }
```

```

    }
# Define a callback to record evaluation metrics
class MetricsCallback(TrainerCallback):
    def __init__(self):
        self.training_loss = []
        self.eval_loss = []

    def on_log(self, args, state, control, logs=None, **kwargs):
        if 'loss' in logs:
            self.training_loss.append(logs['loss'])
        if 'eval_loss' in logs:
            self.eval_loss.append(logs['eval_loss'])
# Initialize the callback
metrics_callback = MetricsCallback()
# Load datasets
train_dataset = HRDataset(tokenizer, '/content/hr_train.csv') # Update path
val_dataset = HRDataset(tokenizer, '/content/hr_val.csv') # Update path

# Setting up training arguments
training_args = TrainingArguments(
    output_dir="./gpt2-finetuned-hr",
    num_train_epochs=7, # Increased from 0.5 to 3
    per_device_train_batch_size=4, # Adjust based on your GPU capability
    per_device_eval_batch_size=4, # Adjust based on your GPU capability
    warmup_steps=300, # Adjusted based on the dataset size and learning rate
    weight_decay=0.01,
    logging_dir='./logs',
    logging_steps=10,
    evaluation_strategy="steps", # Evaluate the model every 'eval_steps'
    eval_steps=500, # Evaluate the model every 500 steps
    save_strategy="steps", # Save the model checkpoint every 'save_steps'
    save_steps=500, # Save the model checkpoint every 500 steps
    learning_rate=5e-5 # Adjust if necessary after observing training behavior
)

# Initializing the Trainer
trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=train_dataset, # Make sure train_dataset is defined and
    ↪loaded
    eval_dataset=val_dataset,
    callbacks=[metrics_callback] # Make sure val_dataset is defined and loaded
)

# Start training
trainer.train()

```

```

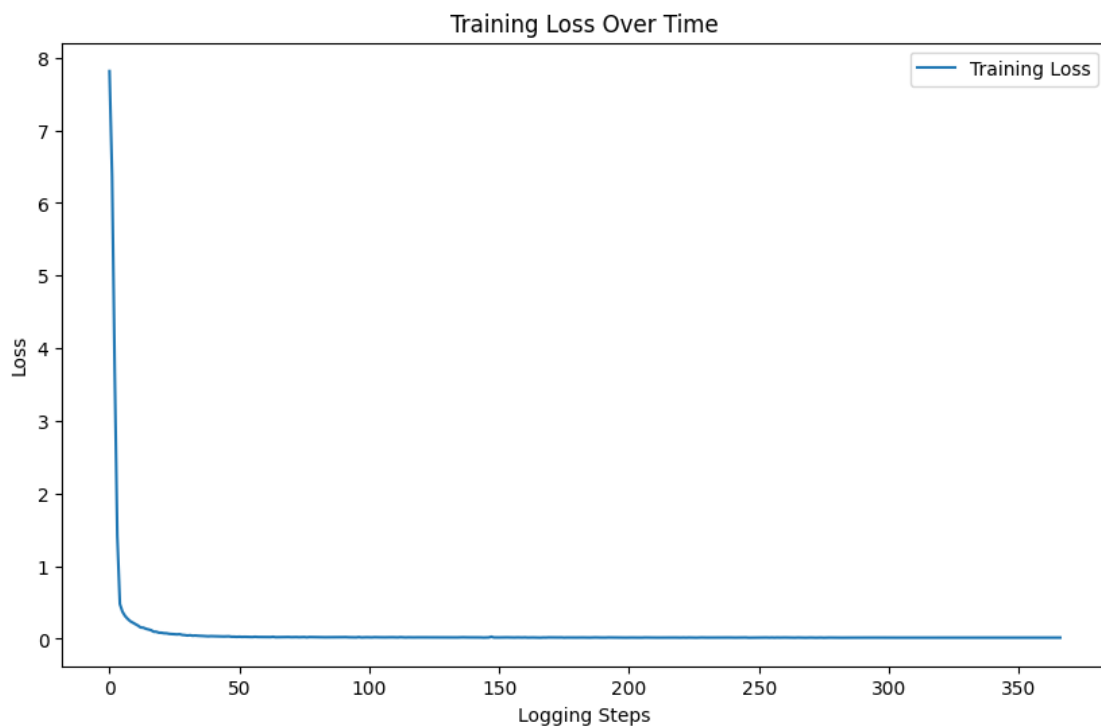
# After training, plot the training loss
plt.figure(figsize=(10, 6))
plt.plot(metrics_callback.training_loss, label='Training Loss')
plt.xlabel('Logging Steps')
plt.ylabel('Loss')
plt.title('Training Loss Over Time')
plt.legend()
plt.show()

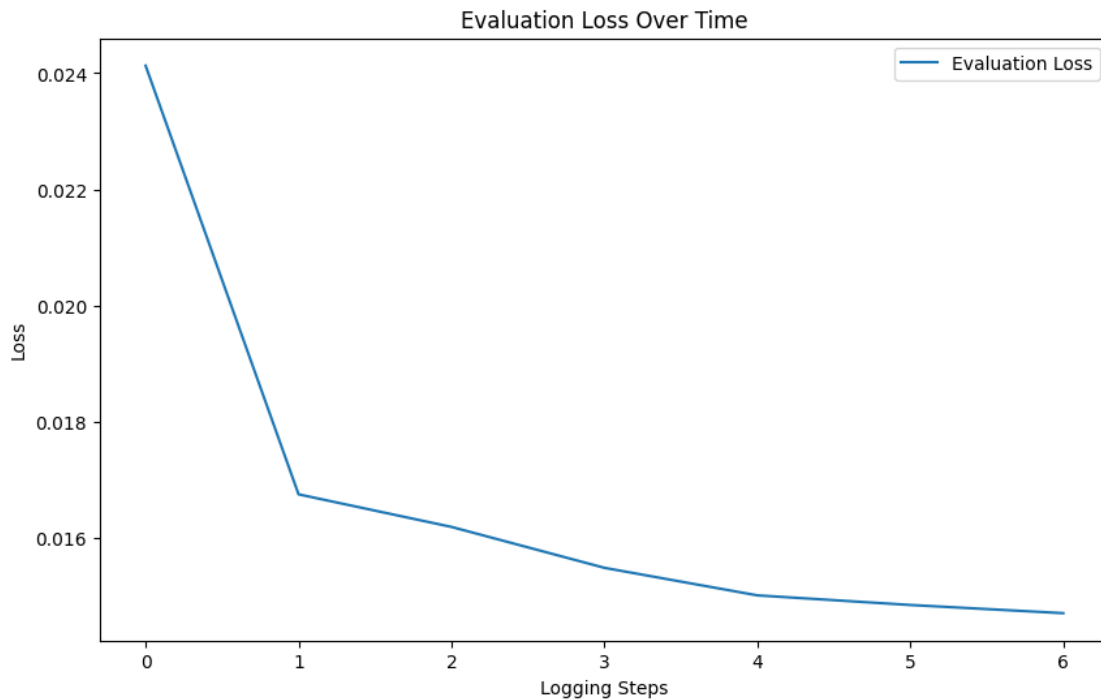
# After training, plot the evaluation loss (if any)
if metrics_callback.eval_loss:
    plt.figure(figsize=(10, 6))
    plt.plot(metrics_callback.eval_loss, label='Evaluation Loss')
    plt.xlabel('Logging Steps')
    plt.ylabel('Loss')
    plt.title('Evaluation Loss Over Time')
    plt.legend()
    plt.show()

# Save the fine-tuned model
model.save_pretrained("./gpt2-finetuned-hr")
tokenizer.save_pretrained('gpt2-finetuned-hr')

```

<IPython.core.display.HTML object>





```
[6]: ('gpt2-finetuned-hr/tokenizer_config.json',
      'gpt2-finetuned-hr/special_tokens_map.json',
      'gpt2-finetuned-hr/vocab.json',
      'gpt2-finetuned-hr/merges.txt',
      'gpt2-finetuned-hr/added_tokens.json')
```

9 Tensorboard Section-More Fine-Tuning Metrics

```
[7]: %load_ext tensorboard
      %tensorboard --logdir ./logs
      import matplotlib.pyplot as plt

      training_loss = [] # List to store training loss per logging step

      # Assuming you have a function that appends the loss to training_loss
      # during training, which could be a custom callback

      # After training, plot the training loss
      plt.figure(figsize=(10, 6))
      plt.plot(training_loss, label='Training Loss')
      plt.xlabel('Logging Steps')
      plt.ylabel('Loss')
```

```

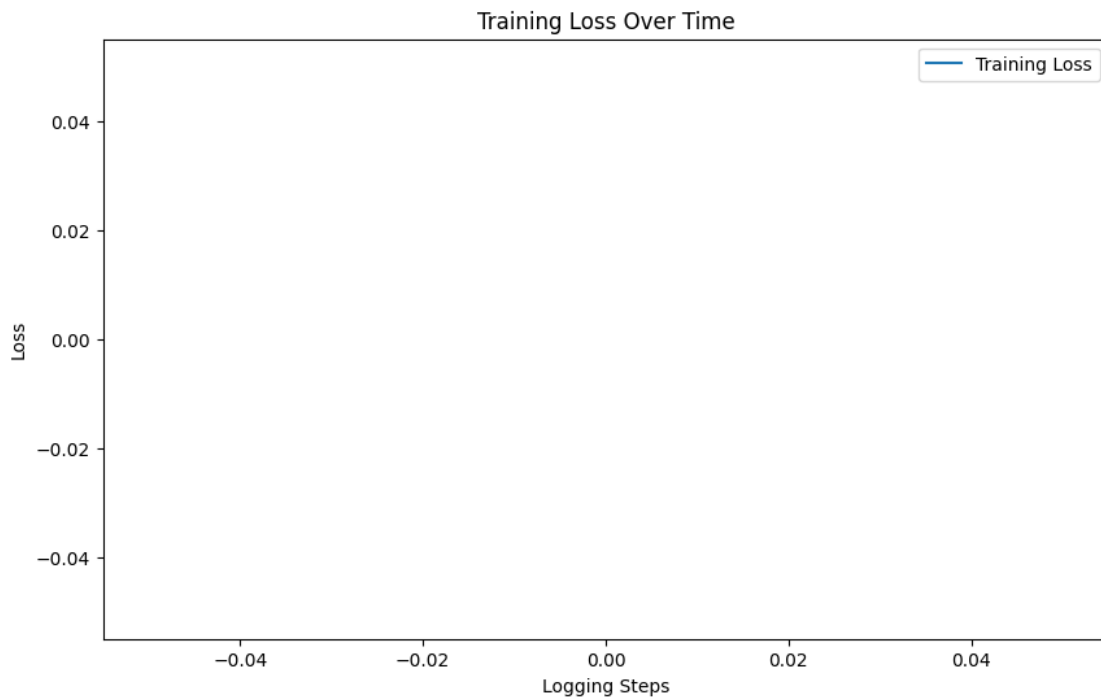
plt.title('Training Loss Over Time')
plt.legend()
plt.show()
eval_accuracy = [] # List to store evaluation accuracy per eval step

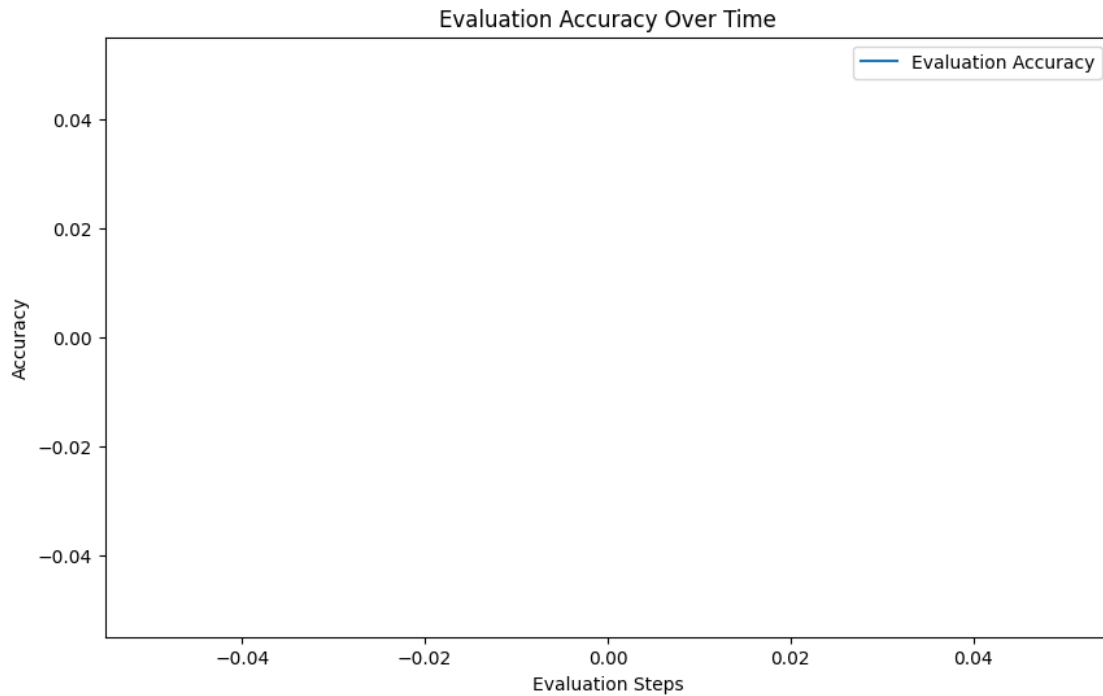
# Similar to loss, store the evaluation metrics during training

# After training, plot the evaluation accuracy
plt.figure(figsize=(10, 6))
plt.plot(eval_accuracy, label='Evaluation Accuracy')
plt.xlabel('Evaluation Steps')
plt.ylabel('Accuracy')
plt.title('Evaluation Accuracy Over Time')
plt.legend()
plt.show()

```

<IPython.core.display.Javascript object>





10 Final Fine-Tuned Neural Network Test and Evaluation-Test By examples

```
[8]: from tqdm.auto import tqdm

# Load the fine-tuned model and tokenizer
model_path = '/content/gpt2-finetuned-hr' # Update this path to your model's location
tokenizer = GPT2Tokenizer.from_pretrained('/content/gpt2-finetuned-hr')
# Load the Excel file
file_path = '/content/biasData4.xlsx' # Update with the correct path if needed
df = pd.read_excel(file_path)
df_first_100 = df.head(800).copy()
# Function to generate HR recommendations
def generate_hr_recommendation(text, model, tokenizer, max_length=100):
    # Format the input text as a prompt similar to the training format
    prompt = f"Manager Comment: {text} HR_Insight:"

    # Encode the input text
    input_ids = tokenizer.encode(prompt, return_tensors='pt')
    input_ids = input_ids.to(model.device)

    # Generate the output with adjusted parameters
```

```

output_ids = model.generate(
    input_ids,
    max_length=max_length,
    num_return_sequences=1,
    pad_token_id=tokenizer.eos_token_id,
    no_repeat_ngram_size=2
)

# Decode the generated text
generated_text = tokenizer.decode(output_ids[0], skip_special_tokens=True)

# Extract only the generated part (remove the input prompt)
generated_part = generated_text[len(prompt):].strip()
return generated_part

# Prepare the data for prediction
#manager_comments = df_first_100['Manager Comment']
manager_comments = df_first_100['Manager Comment']

# Generate HR recommendations
hr_recommendations = [generate_hr_recommendation(comment, model, tokenizer) for
    ↪comment in tqdm(manager_comments, desc="Generating HR Recommendations")]

# Add the generated recommendations to the dataframe
df_first_100.loc[:, 'Test_Recommendations'] = hr_recommendations
# Save the updated DataFrame back to an Excel file
output_file_path = '/content/Updated_HR_Recommendations_3.xlsx' # Update with
    ↪the desired output path
df_first_100.to_excel(output_file_path, index=False)

# Display the updated dataframe
df_first_100.head()

```

```

[8]:
      Employee Name Ethnicity  Age  Years of Experience  \
0  Kostas Angelopoulos (205067)   Greek    33             14
1    Grace Miller (165818)   English    48             14
2    Eleni Dimitriou (463543)   Greek    29              4
3    Sofia Kostas (935433)   Greek    24              3
4    Rohan Singh (892070)   Indian    27              2

      Manager Comment  \
0  Kostas Angelopoulos displays a strong work eth...
1  Grace Miller is exceptional in mentoring and c...
2  Eleni Dimitriou shows outstanding problem-solv...

```


3 Sofia Kostas has a keen eye for innovation and...
 4 Rohan Singh excels in delivering detailed and ...

	Employee Performance Review	Position \
0	Kostas Angelopoulos consistently shows dedicat...	Business Consultant
1	Grace Miller consistently leads by example and...	Graphic Designer
2	Eleni Dimitriou demonstrates exceptional profi...	Research Scientist
3	Sofia Kostas is a driving force behind innovat...	Research Scientist
4	Rohan Singh shows innovative problem-solving u...	Project Manager

	Department \
0	Business Analysis
1	Design
2	Research
3	Research
4	Technical Program Management

Test_Recommendations

0 your strong working environment and dedication...
 1 your ability to guide and coach others is admi...
 2 your knack for solving complex problems is com...
 3 your ability to identify areas for improvement...
 4 your ability to deliver detailed work consiste...

```
[14]: # Manually provided manager comment
manager_comment = "John has consistently met his targets and has shown
↳excellent teamwork."

# Generate HR recommendation
hr_recommendation = generate_hr_recommendation(manager_comment, model,
↳tokenizer)
print("HR Recommendation:", hr_recommendation)

# Manually provided manager comment
manager_comment = "Andreas lacks technical skills."

# Generate HR recommendation
hr_recommendation = generate_hr_recommendation(manager_comment, model,
↳tokenizer)
print("HR Recommendation:", hr_recommendation)
```

HR Recommendation: your teamwork is highly valued john we encourage you to keep up the great work

HR Recommendation: to improve your technical expertise participating in our upcoming technology workshop would be highly beneficial

```
[23]: import os
import shutil

def remove_contents_of_directory(directory, exclude=None):
    if exclude is None:
        exclude = []

    for item in os.listdir(directory):
        item_path = os.path.join(directory, item)
        if item in exclude:
            print(f"Skipping: {item_path}")
            continue # Skip the excluded items

        if os.path.isdir(item_path):
            shutil.rmtree(item_path)
            print(f"Removed directory: {item_path}")
        elif os.path.isfile(item_path):
            os.remove(item_path)
            print(f"Removed file: {item_path}")

# Directory to clean
directory_to_clean = '/content'

# Directory to exclude from deletion
exclude_dir = ['drive']

# Remove contents of '/content' directory except the 'drive' folder
remove_contents_of_directory(directory_to_clean, exclude=exclude_dir)
```

```
Removed directory: /content/.config
Removed file: /content/hr_val.csv
Removed file: /content/biasData4.xlsx
Removed file: /content/Updated_HR_Recommendations_3.xlsx
Removed file: /content/hr_train.csv
Skipping: /content/drive
Removed file: /content/HR_Recommendations.xlsx
Removed file: /content/hr_test.csv
Removed directory: /content/sample_data
```

```
[19]: from google.colab import drive
drive.mount('/content/drive')
```

Mounted at /content/drive

Copy From content to Drive

```
[20]: import shutil
import os
```

```

destination = '/content/drive/MyDrive/MasterNeapolis/training_data'

if not os.path.exists(destination):
    os.makedirs(destination)

for item in os.listdir('/content'):
    s = os.path.join('/content', item)
    d = os.path.join(destination, item)
    if os.path.isdir(s) and item != 'drive':
        shutil.copytree(s, d, dirs_exist_ok=True)
    elif os.path.isfile(s):
        shutil.copy2(s, d)

```

Copy the training Data

```

[24]: import shutil
import os

# Set the source path to the 'training_data' directory in Google Drive
source = '/content/drive/MyDrive/MasterNeapolis/training_data'

# Set the destination path to the '/content' directory
destination = '/content'

# Check if the source directory exists before attempting to copy
if os.path.exists(source):
    for item in os.listdir(source):
        s = os.path.join(source, item)
        d = os.path.join(destination, item)
        # Check if it's a file or directory and copy accordingly
        if os.path.isdir(s):
            shutil.copytree(s, d, dirs_exist_ok=True) # Copy directories
        elif os.path.isfile(s):
            shutil.copy2(s, d) # Copy files
else:
    print(f"Source directory {source} does not exist.")

```